Machine Translated by Google

The Complete Reference



الفصل الحادي عشر

نظرة عامة على C++

255

جميع الحقوق محفوظة ٢٠٠٣ ©لشركة ماكجرو هيل. انقر هنا للاطلاع على شروط الاستخدام.

Machine Translated by Google

لغة برمجة كائنية التوجه، وخصائصها الكائنية مترابطة بشكل كبير. في كثير من الأحيان، يُصعّب هذا الترابط وصف ميزة واحدة من لغة C++ دون أن يشمل ضمنيًا العديد من الميزات الأخرى. علاوة على ذلك، يستكشف كتاب C++ الكائنية التوجه، في كثير من الأحيان، لدرجة أن مناقشة إحدى الميزات تستلزم معرفة مسبقة بأخرى. لمعالجة هذه المشكلة، يُقدّم هذا الفصل لمحة عامة سريعة عن أهم جوانب لغة C++، بما في ذلك تاريخها، وخصائصها الرئيسية، والفرق بين C++التقليدية و+ C القياسية. تتناول الفصول المتبقية لغة C++بالتفصيل.

يقدم هذا الفصل نظرة عامة على المفاهيم الأساسية المضمنة في لغة C++، بما في ذلك

أصول لغة C++

بدأت لغة C++كنسخة موسعة من لغة C. اخترع بيارن ستروستروب امتدادات C++لأول مرة عام ١٩٧٩في مختبرات بيل في موراي هيل، نيو جيرسي. أطلق على اللغة الجديدة في البداية اسم C"مع الفئات". ولكن في عام ١٩٨٣، تم تغيير الاسم إلى C++.

على الرغم من أن لغة C كانت من أكثر لغات البرمجة الاحترافية شيوعًا واستخدامًا في العالم، إلا أن اختراع C++كان ضروريًا بسبب عامل برمجي رئيسي واحد: تزايد التعقيد. على مر السنين، أصبحت برامج الحاسوب أكبر حجمًا وأكثر تعقيدًا. ورغم أن C لغة برمجة ممتازة، إلا أن لها حدودها. ففي C، بمجرد أن يتجاوز البرنامج 25,000 إلى 100,000 سطر من التعليمات البرمجية، يصبح معقدًا للغاية بحيث يصعب فهمه ككل. والغرض من C++هو كسر هذا الحاجز. جوهر C++هو تمكين المبرمج من فهم وإدارة برامج أكبر وأكثر تعقيدًا.

تدعم معظم الإضافات التي أدخلها ستروستروب على لغة C البرمجة الكائنية التوجه، والتي تُعرف أحيانًا باسم OOP. (انظر القسم التالي لشرح موجز للبرمجة الكائنية التوجه). يذكر ستروستروب أن بعض ميزات البرمجة الكائنية التوجه في لغة C++مستوحاة من لغة أخرى كائنية التوجه تُسمى Simula67. لذا، تُمثل C++مزيجًا من طريقتي برمجة قويتين.

منذ اختراع لغة C++، خضعت لثلاث مراجعات رئيسية، كل منها أضافت إليها وغيّرتها. كانت المراجعة الأولى عام ١٩٨٥ والثانية عام ١٩٩٠. أما المراجعة الثالثة، فقد حدثت أثناء توحيد معايير C++قبل عدة سنوات، بدأ العمل على معيار لـ C++ولتحقيق هذه الغاية، شُكِّلت لجنة توحيد معايير مشتركة بين المعهد الوطني الأمريكي للمعايير (ANSI) والمنظمة الدولية للمعايير (ISO). وُضعت المسودة الأولى للمعيار المقترح في ٢٥ يناير ١٩٩٤. في تلك المسودة، احتفظت لجنة C++التابعة لـ ANSI/ISO (والتي كنت عضوًا فيها) بالميزات التي حددها ستروستروب أولًا، وأضافت بعض الميزات الجديدة أيضًا. ولكن بشكل عام، عكست هذه المسودة الأولية حالة C++في ذلك الوقت.

بعد فترة وجيزة من الانتهاء من المسودة الأولى لمعيار C++، حدث ما أدى إلى توسع كبير في اللغة: إنشاء مكتبة القوالب القياسية (STL) بواسطة ألكسندر ستيبانوف. STL هي مجموعة من الإجراءات العامة التي يمكنك استخدامها لمعالجة البيانات. إنها قوية وأنيقة، ولكنها كبيرة الحجم أيضًا. لاحقًا

Machine Translated by Google

في المسودة الأولى، صوّتت اللجنة على إدراج STLضمن مواصفات لغة .C++وقد وسّع إدراج STLنطاق لغة C++بشكل كبير متجاوزًا تعريفها الأصلي. ورغم أهميته، إلا أن إدراج STL، من بين أمور أخرى، أبطأ وتيرة توحيد معايير لغة .C++

من الإنصاف القول إن توحيد لغة C++استغرق وقتًا أطول بكثير مما كان متوقعًا عند بدايته. وخلال هذه العملية، أُضيفت العديد من الميزات الجديدة إلى اللغة، وأُجريت عليها العديد من التغييرات الطفيفة. في الواقع، كانت نسخة C++التي حددتها لجنة C++أكبر وأكثر تعقيدًا بكثير من تصميم ستروستروب الأصلي. وقد أُقرت المسودة النهائية من اللجنة في 14نوفمبر ،1997وأصبح معيار ANSI/ISOللغة C++واقعًا ملموسًا في عام .1998ويُشار إلى هذه المواصفة الخاصة بـ C++عادةً باسم C++القياسي.

تصف المادة الموجودة في هذا الكتاب لغة C++القياسية، بما في ذلك أحدث إصداراتها
الميزات. هذه هي النسخة من C++التي أنشأتها لجنة معايير ،ANSI/ISOوهي النسخة التي يتم قبولها حاليًا من قبل جميع المترجمين الرئيسيين.

ما هي البرمجة الشيئية؟

بما أن البرمجة الكائنية التوجه (OOP)هي التي قادت إلى ظهور لغة ،C++فمن الضروري فهم مبادئها الأساسية. تُعدّ البرمجة الكائنية التوجه (OOP)وسيلة فعّالة للتعامل مع مهمة البرمجة. وقد تغيرت منهجيات البرمجة بشكل كبير منذ اختراع الحاسوب، لا سيما لاستيعاب التعقيد المتزايد للبرامج. على سبيل المثال، عند اختراع الحاسوب لأول مرة، كانت البرمجة تُنفّذ عن طريق تشغيل تعليمات الآلة الثنائية باستخدام اللوحة الأمامية. وطالما أن البرامج كانت تتكون من بضع مئات من التعليمات فقط، فقد نجح هذا النهج.

مع نمو البرامج، تم اختراع لغة التجميع لتمكين المبرمج من التعامل مع برامج أكبر وأكثر تعقيدًا، باستخدام تمثيلات رمزية لتعليمات الآلة. ومع استمرار نمو البرامج، ظهرت لغات برمجة عالية المستوى أتاحت للمبرمج أدوات أكثر للتعامل مع التعقيد.

كانت لغة فورتران، بالطبع، أول لغة منتشرة على نطاق واسع. ورغم أن فورتران كانت خطوة أولى مبهرة، إلا أنها لا تشجع على تطوير برامج واضحة وسهلة الفهم.

شهدت ستينيات القرن العشرين ظهور البرمجة المنظمة. هذه هي الطريقة التي شجعها
لغات برمجة مثل .Pascalو Cمكّن استخدام اللغات المهيكلة من كتابة برامج متوسطة التعقيد بسهولة نسبية. تتميز هذه اللغات بدعمها للبرامج الفرعية المستقلة، والمتغيرات المحلية، وبنى التحكم الغنية، وعدم اعتمادها على .GOTOعلى الرغم من قوة هذه اللغات، إلا أنها تصل إلى حدودها القصوى عندما يصبح المشروع ضخمًا جدًا.

فكر في هذا: في كل مرحلة مهمة في تطوير البرمجة، يتم تطوير التقنيات
أُنشئت أدواتٌ تُمكّن المبرمج من التعامل مع تعقيداتٍ متزايدة. في كل خطوة، أخذ النهج الجديد أفضل عناصر الطرق السابقة وتقدّم. قبل اختراع البرمجة الكائنية التوجه، كانت العديد من المشاريع تقترب (أو تتجاوز) من النقطة التي لم يعد فيها النهج المنظم صالحًا.

Machine Translated by Google

لقد تم إنشاء طرق موجهة نحو الكائنات لمساعدة المبرمجين على اختراق هذه الحواجز.

استوحت البرمجة كائنية التوجه أفضل أفكار البرمجة الهيكلية ودمجتها مع عدة مفاهيم جديدة. وكانت النتيجة طريقة مختلفة لتنظيم البرنامج. بشكل عام، يمكن تنظيم البرنامج بإحدى طريقتين: حول شيفرته (ما يحدث) أو حول بياناته (من يتأثر). باستخدام تقنيات البرمجة الهيكلية فقط، تُنظم البرامج عادةً حول شيفرته. يمكن اعتبار هذا النهج "شيفرة تؤثر على البيانات".

على سبيل المثال، يتم تعريف البرنامج المكتوب بلغة منظمة مثل Cمن خلال وظائفه، والتي يمكن لأي منها أن تعمل على أي نوع من البيانات التي يستخدمها البرنامج.
تعمل البرامج كائنية التوجه بطريقة معاكسة. فهي تُنظّم حول البيانات، والمبدأ الأساسي هو "البيانات التي تتحكم في الوصول إلى الشيفرة". في لغة البرمجة كائنية التوجه، تُعرّف البيانات والبرامج المسموح لها بالعمل عليها. وبالتالي، يُحدد نوع البيانات بدقة نوع العمليات التي يُمكن تطبيقها على تلك البيانات.

لدعم مبادئ البرمجة الكائنية التوجه، تشترك جميع لغات البرمجة الكائنية التوجه في ثلاث سمات: التغليف، وتعدد الأشكال، والوراثة. لنتفحص كل سمة.

التغليف هو الآلية التي تربط الكود والبيانات التي يعالجها، وتحميهما من التدخل الخارجي وسوء الاستخدام. في لغة البرمجة كائنية التوجه، يمكن دمج الكود والبيانات بطريقة تُنشئ "صندوقًا أسود" مستقلًا. عند ربط الكود والبيانات بهذه الطريقة، يُنشأ كائن . بمعنى آخر، الكائن هو الجهاز الذي يدعم التغليف.

داخل الكائن، قد يكون الكود أو البيانات أو كليهما خاصًا بذلك الكائن أو عامًا. خاص
الشيفرة أو البيانات معروفة ولا يمكن الوصول إليها إلا من قِبل جزء آخر من الكائن. أي أنه لا يمكن الوصول إلى الشيفرة أو البيانات الخاصة بواسطة جزء من البرنامج موجود خارج الكائن. عندما تكون الشيفرة أو البيانات عامة، يمكن لأجزاء أخرى من برنامجك الوصول إليها حتى لو كانت مُعرّفة داخل الكائن. عادةً، تُستخدم الأجزاء العامة من الكائن لتوفير واجهة مُتحكّم بها لعناصره الخاصة.

لجميع الأغراض والمقاصد، الكائن هو متغير من نوع مُعرّف من قِبل المستخدم. قد
قد يبدو غريبًا أن يُعتبر الكائن الذي يربط بين الكود والبيانات متغيرًا. مع ذلك، في البرمجة الكائنية التوجه، هذا هو الحال تمامًا. في كل مرة تُعرّف فيها نوعًا جديدًا من الكائنات، تُنشئ نوعًا جديدًا من البيانات. كل مثيل محدد من هذا النوع من البيانات هو متغير مركب.

تدعم لغات البرمجة كائنية التوجه تعدد الأشكال، والذي يُعرّف بعبارة "واجهة واحدة، طرق متعددة". ببساطة، تعدد الأشكال هو الخاصية التي تسمح لواجهة واحدة بالتحكم في الوصول إلى فئة عامة من الإجراءات.

Machine Translated by Google

يُحدَّد الإجراء المُحدَّد بناءً على طبيعة الموقف. ومن الأمثلة الواقعية على تعدد الأشكال منظم الحرارة (الثرموستات). فبغض النظر عن نوع الفرن في منزلك (غاز، زيت، كهرباء، إلخ)، يعمل منظم الحرارة بالطريقة نفسها. في هذه الحالة، يكون منظم الحرارة (الذي يُمثِّل الواجهة) هو نفسه بغض النظر عن نوع الفرن (الطريقة) المُستخدم.

على سبيل المثال، إذا كنت تريد درجة حرارة 70درجة، يمكنك ضبط منظم الحرارة على 70درجة.
لا يهم نوع الفرن الذي يوفر الحرارة فعليًا.
يمكن تطبيق هذا المبدأ نفسه على البرمجة. على سبيل المثال، قد يكون لديك برنامج يُعرّف ثلاثة أنواع مختلفة من المكدسات.
يُستخدم أحدها لقيم الأعداد الصحيحة، والآخر لقيم الأحرف، والثالث لقيم الفاصلة العائمة. بفضل تعدد الأشكال، يمكنك تعريف مجموعة واحدة من الأسماء، ( )pushو ،( )popوالتي يمكن استخدامها لجميع المكدسات الثلاثة. في برنامجك، ستُنشئ ثلاثة إصدارات محددة من هذه الدوال، إصدار لكل نوع من أنواع المكدسات، ولكن أسماء الدوال ستكون متطابقة. سيختار المُجمّع تلقائيًا الدالة المناسبة بناءً على البيانات المُخزّنة.

وبالتالي، فإن واجهة المكدس -الدالتان ( )pushو - ( )popهي نفسها بغض النظر عن نوع المكدس المُستخدم. تُحدد الإصدارات الفردية لهذه الدالات التطبيقات (الأساليب) الخاصة بكل نوع من البيانات.

يساعد تعدد الأشكال على تقليل التعقيد من خلال السماح باستخدام نفس الواجهة
للوصول إلى فئة عامة من الإجراءات. تقع على عاتق المُجمِّع مهمة اختيار الإجراء المُحدد (أي الطريقة) حسب تطبيقه في كل حالة.
لستَ مُضطرًا، كمُبرمج، إلى القيام بهذا الاختيار يدويًا. ما عليك سوى تذكر الواجهة العامة واستخدامها.

كانت أولى لغات البرمجة كائنية التوجه تعتمد على المفسّرات، لذا كان تعدد الأشكال مدعومًا أثناء التشغيل. أما لغة C++فهي لغة مُجمّعة. لذلك، تدعم C++تعدد الأشكال أثناء التشغيل وأثناء التجميع.

الميراث

الوراثة هي العملية التي يكتسب بها كائن خصائص كائن آخر. تُعد هذه العملية مهمة لأنها تدعم مفهوم التصنيف. إذا تأملت الأمر، ستجد أن معظم المعرفة تُصبح قابلة للإدارة من خلال التصنيفات الهرمية. على سبيل المثال، تُعتبر تفاحة Red Deliciousجزءًا من تفاحة التصنيف، والتي بدورها جزء من فئة الفاكهة ، والتي تندرج تحت فئة الطعام الأكبر. بدون استخدام التصنيفات، سيتعين على كل كائن تعريف جميع خصائصه صراحةً. ومع ذلك، من خلال استخدام التصنيفات، لا يحتاج الكائن إلا إلى تعريف الصفات التي تجعله فريدًا داخل فئته. إنها آلية الوراثة التي تُمكّن كائنًا واحدًا من أن يكون مثالًا محددًا لحالة أكثر عمومية. كما سترى، يُعد الوراثة جانبًا مهمًا من البرمجة الكائنية التوجه.

بعض أساسيات لغة :C++في الجزء الأول، وُصفت مجموعة Cالفرعية من لغة ،C++واستُخدمت برامج Cلتوضيح هذه الميزات. من الآن فصاعدًا، ستكون جميع الأمثلة ."C++"

Machine Translated by Google

البرامج." أي أنهم سيستخدمون ميزات فريدة لـ .++Cلتسهيل المناقشة، سنشير إلى هذه الميزات الخاصة بـ ++Cببساطة باسم "ميزات "++Cمن
الآن.

إذا كنتَ من خلفية لغة ،Cأو كنتَ تدرس برامج Cالفرعية في الجزء الأول، فاعلم أن برامج ++Cتختلف عن برامج Cفي بعض الجوانب المهمة. تتعلق معظم هذه الاختلافات بالاستفادة من إمكانيات ++Cالكائنية التوجه. إلا أن برامج ++Cتختلف عنها في جوانب أخرى، بما في ذلك كيفية إجراء عمليات الإدخال والإخراج والرؤوس المُضمنة. كما تشترك معظم برامج ++Cفي مجموعة من السمات المشتركة التي تُميّزها بوضوح كبرامج .++C

قبل الانتقال إلى بنيات ++Cالموجهة للكائنات، مطلوب فهم العناصر الأساسية لبرنامج .++C

يصف هذا القسم العديد من القضايا المتعلقة بجميع برامج ++Cتقريبًا.
في هذه المقالة، تم الإشارة إلى بعض الاختلافات المهمة بين لغة Cوالإصدارات السابقة من لغة .++C

برنامج ++Cنموذجي لنبدأ بالبرنامج ++Cالنموذجي القصير الموضح هنا.

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

int main() {

  int i;

  cout << "هذا هو الإخراج. // ";\n"هذا تعليق على سطر واحد / *لا يزال بإمكانك استخدام تعليقات النمط /* C

  //أدخل رقمًا باستخدام cout << "أدخل رقمًا: "; cin >> i;

  //الآن، قم بإخراج رقم باستخدام ;"\n" << i*i << << <<
  cout << i << "التربيع هو"

  العودة 0؛
}
```

كما ترى، يبدو هذا البرنامج مختلفًا تمامًا عن برامج Cالفرعية المذكورة في الجزء الأول. سيكون شرح كل سطر مفيدًا. للبدء، تم تضمين العنوان . <iostream>يدعم هذا العنوان عمليات الإدخال/الإخراج بنمط <iostream>) .++Cفي ++Cهو بمثابة stdio.hفي .(Cلاحظ شيئًا آخر: لا يوجد امتداد .h لـ

Machine Translated by Google

الاسم .iostreamالسبب هو أن <iostream>هو أحد العناوين الحديثة المُعرّفة بلغة C++القياسية. لا تستخدم عناوين C++ الحديثة امتداد .h .
السطر التالي في البرنامج هو



استخدام مساحة الاسم ؛std

هذا يُخبر المُجمِّع باستخدام مساحة الأسماء القياسية . (std)تُعدّ مساحات الأسماء إضافةً حديثةً إلى لغة .C++تُنشئ مساحة الأسماء منطقةً تعريفيةً يُمكن وضع عناصر البرنامج فيها. تُساعد مساحات الأسماء في تنظيم البرامج الكبيرة. تُعلم عبارة الاستخدام المُجمِّع برغبتك في استخدام مساحة الأسماء القياسية . (std)هذه هي مساحة الأسماء التي تُعلن فيها مكتبة C++القياسية بالكامل. باستخدام مساحة الأسماء القياسية، تُبسِّط الوصول إلى المكتبة القياسية. لا تحتاج البرامج في الجزء الأول، والتي تستخدم مجموعة Cالفرعية فقط، إلى جملة مساحة الأسماء لأن دوال مكتبة Cمتاحة أيضًا في مساحة الأسماء العامة الافتراضية.

Note

بما أن كلاً من الرؤوس ومساحات الأسماء الجديدة هي إضافات حديثة إلى لغة ،C++فقد تواجه أكوادًا قديمة لا تستخدمها. كذلك، إذا كنت تستخدم مُجمِّعًا قديمًا، فقد لا يدعمها. ستجد تعليمات استخدام مُجمِّع قديم لاحقًا في هذا الفصل.

الآن قم بفحص السطر التالي.



int main()

لاحظ أن قائمة المعاملات في ( )mainفارغة. في ،C++يشير هذا إلى أن ( )mainلا تحتوي على أي معاملات. وهذا يختلف عن .Cفي ،Cيجب على الدالة التي لا تحتوي على معاملات استخدام voidفي قائمة معاملاتها، كما هو موضح هنا:



int main(void)

هكذا تم الإعلان عن ( )mainفي برامج الجزء الأول. مع ذلك، في لغة ،C++يُعد استخدام voidزائدًا عن الحاجة. كقاعدة عامة، في ،C++عندما لا تأخذ الدالة أي معاملات، تكون قائمة معاملاتها فارغة؛ لذا لا حاجة لاستخدام . void

يحتوي السطر التالي على ميزتين من ميزات .C++



>> cout"هذا هو الإخراج. // ";"\nهذا تعليق بسطر واحد

أولا البيان



>> cout"هذا هو الإخراج.;"\n\

Machine Translated by Google

يؤدي هذا إلى عرض هذا الناتج على الشاشة، متبوعًا بمزيج من إرجاع السطر وتغذية السطر. في لغة C++، للعلامة >> دور موسّع. لا تزال تُشغّل عامل الإزاحة لليسار، ولكن عند استخدامها كما هو موضح في هذا المثال، تُعدّ أيضًا عامل إخراج. كلمة cout هي مُعرّف مرتبط بالشاشة. (في الواقع، كما هو الحال في لغة C++، تدعم إعادة توجيه الإدخال/الإخراج، ولكن للتوضيح، افترض أن cout تُشير إلى الشاشة).

يمكنك استخدام cout و >> لإخراج أي من أنواع البيانات المضمنة، بالإضافة إلى سلاسل الأحرف.

لاحظ أنه لا يزال بإمكانك استخدام printf( ) أو أي دالة إدخال/إخراج أخرى في لغة C++ في برنامج C++. مع ذلك، يرى معظم المبرمجين أن استخدام >> أقرب إلى لغة C++.
علاوة على ذلك، في حين أن استخدام printf( ) لإخراج سلسلة يعادل تقريبًا استخدام >> في هذه الحالة، يمكن توسيع نظام الإدخال/الإخراج + C++ لإجراء عمليات على الكائنات التي تحددها (وهو شيء لا يمكنك القيام به باستخدام printf( )).

ما يلي تعبير المخرجات هو تعليق سطري واحد في لغة C++. كما ذكرنا في الفصل العاشر، تُعرّف لغة C++ نوعين من التعليقات. أولًا، يمكنك استخدام تعليق متعدد الأسطر، وهو يعمل في C++ بنفس الطريقة كما في C. يمكنك أيضًا تعريف تعليق سطري واحد باستخدام //; يتجاهل المُجمّع أي تعليق يليه حتى نهاية السطر. بشكل عام، يستخدم مبرمجو C++ التعليقات متعددة الأسطر عند إنشاء تعليق أطول، ويستخدمون التعليقات سطرًا واحدًا عند الحاجة إلى ملاحظة قصيرة فقط.

بعد ذلك، يطلب البرنامج من المستخدم إدخال رقم. يُقرأ الرقم من لوحة المفاتيح بهذه العبارة:

```
cin >> i;
```

في لغة C++، لا يزال عامل >> يحتفظ بمعناه الإزاحة لليمين. ومع ذلك، عند استخدامه كما هو موضح، فهو أيضًا مُعامل إدخال في لغة C++. تُؤدي هذه العبارة إلى قراءة قيمة i من لوحة المفاتيح. يشير المُعرّف cin إلى جهاز الإدخال القياسي، وهو عادةً لوحة المفاتيح. بشكل عام، يُمكنك استخدام >> cin لإدخال مُتغير من أي نوع بيانات أساسي بالإضافة إلى سلاسل نصية.

Note

سطر التعليمات البرمجية الموصوف للتو ليس مطبعيًا بشكل خاطئ. تحديدًا، لا يُفترض وجود & قبل i عند إدخال معلومات باستخدام دالة بلغة C مثل scanf( )، يجب عليك تمرير مؤشر صريح إلى المتغير الذي سيستقبل المعلومات. هذا يعني وضع عامل "عنوان" قبل اسم المتغير، &. مع ذلك، نظرًا لطريقة تطبيق عامل >> في C++، لا تحتاج (بل يجب ألا تستخدم) & سبب ذلك موضح في الفصل 13.

مع أن المثال لم يوضح ذلك، يمكنك استخدام أيٍّ من دوال الإدخال المبنية على لغة C، مثل scanf( ) بدلاً من استخدام >>. مع ذلك، وكما هو الحال مع cout، يرى معظم المبرمجين أن cin >> أقرب إلى لغة C++.

يظهر هنا خط آخر مثير للاهتمام في البرنامج:

```
cout << i << "squared is " << i*i << "\n";
```

Machine Translated by Google

بافتراض أن قيمة اهي ،10سيؤدي هذا البيان إلى عرض العبارة 10تربيع يساوي ، 100متبوعةً بإرجاع سطر. كما يوضح هذا السطر، يمكنك تشغيل عدة عمليات إخراج >>معًا.

وينتهي البرنامج بهذه العبارة:

```
العودة ؛0
```

يؤدي هذا إلى إرجاع قيمة صفر إلى عملية الاستدعاء (والتي عادةً ما تكون نظام التشغيل). يعمل هذا بنفس الطريقة في لغة ++C كما في لغة .Cتشير إرجاع قيمة صفر إلى انتهاء البرنامج بشكل طبيعي. يجب الإشارة إلى انتهاء البرنامج بشكل غير طبيعي بإرجاع قيمة غير صفرية. يمكنك أيضًا استخدام القيمتين EXIT_SUCCESSو EXIT_FAILUREإذا أردت.

نظرة عن قرب على مشغلي الإدخال/الإخراج

كما ذُكر سابقًا، عند استخدام عاملي >>و <<في عمليات الإدخال/الإخراج، يُمكنهما التعامل مع أي نوع بيانات مُضمّن في لغة .C++على سبيل المثال، يُدخل هذا البرنامج عددًا عشريًا، ورقمًا مزدوجًا ، وسلسلة نصية، ثم يُخرِجها:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

int main() {

    تعويم ؛f char str[80]؛
    مزدوج ؛d

    cout << "أدخل رقمين عشريين: "; cin >> f >> d;

    cout << "أدخل سلسلة: "; cin >> str;

    cout << f << " " << د >> " " << str;

    العودة ؛0
}
```

عند تشغيل هذا البرنامج، حاول إدخال "هذا اختبار" عند مطالبتك بإدخال السلسلة النصية. عندما يُعيد البرنامج عرض المعلومات التي أدخلتها، ستظهر كلمة "هذا" فقط. لا يُعرض باقي السلسلة النصية لأن عامل التشغيل <<يتوقف عن قراءة المُدخلات عند أول حرف مسافة. وبالتالي، تكون "هذا اختبار"

Machine Translated by Google

لا يقرأها البرنامج أبدًا. يوضح هذا البرنامج أيضًا إمكانية ربط عدة عمليات إدخال في جملة واحدة.

تتعرف مشغلات الإدخال/الإخراج ++Cعلى المجموعة الكاملة من ثوابت أحرف الشرطة المائلة العكسية كما هو موضح في الفصل الثاني. على سبيل المثال، من المقبول تمامًا أن نكتب

```
cout << "A\tB\tC";
```

يقوم هذا البيان بإخراج الأحرف Bو AوCو A، مفصولة بعلامات التبويب.

إعلان المتغيرات المحلية

إذا كنتَ من خلفية C، فعليكَ أن تُدرك فرقًا هامًا بين CوC+ +فيما يتعلق بموعد إعلان المتغيرات المحلية. في C89، يجب إعلان جميع المتغيرات المحلية المستخدمة داخل كتلة في بداية تلك الكتلة. لا يُمكنك إعلان متغير في كتلة بعد حدوث جملة "action"على سبيل المثال، في C89، هذا الجزء غير صحيح:

```
/*غير صحيح في C89. صحيح في C++. */
int f() {

int i; i = 10؛

/* int j; لن يتم تجميعه كبرنامج C */ j = i*2;

العودة ج؛
}
```

في برنامج C89، تكون هذه الدالة خاطئة لأن التعيين يتداخل بين إعلان iوإعلان j. ومع ذلك، عند تجميعها كبرنامج C++، يكون هذا الجزء مقبولًا تمامًا. في C++(وC99)، يمكنك إعلان المتغيرات المحلية في أي نقطة داخل كتلة، وليس فقط في بدايتها.

هذا مثال آخر. هذه النسخة من البرنامج من القسم السابق يعلن strقبل الحاجة إليه مباشرة.

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

int main() {

تعويم f؛
```

Machine Translated by Google


```
>> double d; cout <<"أدخل رقمين عشريين: "; cin >> f >> d;

>> cout <<"أدخل سلسلة: "; // str char str[80]; "تم الإعلان عنها هنا، قبل الاستخدام الأول مباشرةً" ;cin >> str;

cout << f << " " << د >> " " << str;

العودة ؛0
}
```


سواءٌ أكنتَ تُعلن جميع المتغيرات في بداية كتلة أو عند أول استخدام، فالأمر متروك لك تمامًا. بما أن فلسفة C++تعتمد في جزء كبير منها على تغليف الكود والبيانات، فمن المنطقي أن تُعلن المتغيرات بالقرب من مكان استخدامها بدلًا من مجرد بداية الكتلة. في المثال السابق، تم فصل الإعلانات للتوضيح فقط، ولكن من السهل تخيّل أمثلة أكثر تعقيدًا تكون فيها هذه الميزة في C++أكثر قيمة.

إن إعلان المتغيرات بالقرب من المكان الذي يتم استخدامها فيه يمكن أن يساعدك في تجنب الأخطاء الجانبية العرضية التأثيرات. مع ذلك، تُكتسب أكبر فائدة من إعلان المتغيرات عند الاستخدام الأول في الدوال الكبيرة. بصراحة، في الدوال القصيرة (كما في العديد من الأمثلة في هذا الكتاب)، لا يوجد سبب يُذكر لعدم إعلان المتغيرات في بداية الدالة. لهذا السبب، سيُعلن هذا الكتاب عن المتغيرات عند الاستخدام الأول فقط عندما يبدو ذلك مبررًا بحجم الدالة أو تعقيدها.

هناك جدل حول الحكمة العامة من توطين إعلانات المتغيرات. يرى المعارضون أن توزيع الإعلانات في كتلة برمجية واحدة يُصعّب، لا يُسهّل، على قارئ الشيفرة البرمجية العثور بسرعة على إعلانات جميع المتغيرات المستخدمة في تلك الكتلة، مما يُصعّب صيانة البرنامج. لهذا السبب، لا يُوظّف بعض مبرمجي C++هذه الميزة بشكل كبير. لن يُؤيّد هذا الكتاب هذه المسألة. مع ذلك، عند تطبيقها بشكل صحيح، وخاصةً في الدوال الكبيرة، يُمكن أن يُساعدك إعلان المتغيرات عند استخدامها لأول مرة على إنشاء برامج خالية من الأخطاء بسهولة أكبر.

قبل بضع سنوات، طُرح تغيير على لغة C++قد يؤثر على أكواد C++القديمة، بالإضافة إلى أكواد C++التي تُنقل إليها. تنص كلٌّ من C89والمواصفات الأصلية لـ C++على أنه عند عدم تحديد نوع صريح في الإعلان، يُفترض النوع . intمع ذلك، تم حذف قاعدة "الافتراضي إلى "intمن C++أثناء عملية التوحيد القياسي. كما حذفت C99هذه القاعدة أيضًا. ومع ذلك، لا يزال هناك عدد كبير من أكواد C++القديمة و+ Cتستخدم هذه القاعدة.

Machine Translated by Google

الاستخدام الأكثر شيوعًا لقاعدة "الافتراضي إلى عدد صحيح" هو مع أنواع إرجاع الدوال. كان من الشائع عدم تحديد عدد صحيح صراحةً عند إرجاع دالة نتيجة عدد صحيح. على سبيل المثال، في لغة C89 وC+ +الأقدم، الدالة التالية صحيحة.

```
دالة(int i) { i*i؛ العودة
}
```

في لغة C++القياسية، يجب أن يكون لهذه الوظيفة نوع الإرجاع intالمحدد، كما هو موضح هنا.

```
int دالة(int i) { i*i؛ العودة
}
```

عمليًا، لا تزال جميع مُجمِّعات C++تقريبًا تدعم قاعدة "الافتراضي إلى عدد صحيح" للتوافق مع الأكواد القديمة. مع ذلك، لا يُنصح باستخدام هذه الميزة مع الأكواد الجديدة، لأنها لم تعد مسموحة.

نوع البيانات المنطقية

تُعرّف لغة C++نوعًا منطقيًا مُدمجًا يُسمى bool. لا يُمكن للكائنات من نوع boolتخزين سوى القيمتين true وfalse، وهما كلمتان مفتاحيتان مُعرّفتان في C++.كما هو موضح في الجزء الأول، تحدث تحويلات تلقائية تسمح بتحويل قيم boolإلى أعداد صحيحة، والعكس صحيح. تحديدًا، تُحوّل أي قيمة غير صفرية إلى true، ويُحوّل الصفر إلى false. ويحدث العكس أيضًا؛ حيث تُحوّل trueإلى 1، وتُحوّل falseإلى صفر.

وهكذا، فإن المفهوم الأساسي المتمثل في كون الصفر خاطئًا وغير الصفر صحيحًا لا يزال راسخًا بشكل كامل في لغة C++.

Note

مع أن لغة C89(وهي لغة فرعية من لغة C++)لا تُعرّف نوعًا منطقيًا، إلا أن لغة C99تُضيف إلى لغة Cنوعًا يُسمى _Bool، وهو قادر على تخزين القيمتين 1و0 (أي صحيح/خطأ). بخلاف لغة C++، لا تُعرّف لغة C99الكلمتين trueو falseككلمات مفتاحية.

وبالتالي، فإن _Boolكما هو محدد بواسطة C99غير متوافق مع boolكما هو محدد بواسطة C++. السبب في تحديد لغة C99لـ _Boolبدلاً من boolككلمة مفتاحية هو أن العديد من برامج Cالموجودة مسبقًا قد عرّفت إصداراتها المخصصة من bool. بتعريف النوع Booleanكـ _Bool، تتجنب لغة C99خرق هذا الكود الموجود مسبقًا. ومع ذلك، من الممكن تحقيق التوافق بين C++وC99 في هذه النقطة لأن C99تضيف العنوان <stdbool.h>الذي يُعرّف وحدات الماكرو boolو trueو falseبإضافة هذا العنوان، يمكنك إنشاء كود متوافق مع كلٍّ من C99وC+.+

Machine Translated by Google

لغة C++القديمة مقابل لغة C++الحديثة

كما أوضحنا، خضعت لغة C++لعملية تطورية واسعة النطاق خلال تطويرها وتوحيد معاييرها. ونتيجة لذلك، يوجد نسختان منها.

النسخة الأولى هي النسخة التقليدية المبنية على تصاميم بيارن ستروستروب الأصلية. أما النسخة الثانية فهي لغة C++القياسية، التي أنشأها ستروستروب ولجنة التقييس .ANSI/ISOمع أن هاتين النسختين من لغة C++متشابهتان جدًا في جوهرهما، إلا أن لغة C++القياسية تحتوي على العديد من التحسينات غير الموجودة في لغة C++التقليدية. وبالتالي، فإن لغة C++القياسية هي في جوهرها نسخة فرعية من لغة C++التقليدية.

يصف هذا الكتاب لغة C++القياسية. هذه هي نسخة C++التي حددتها لجنة التقييس ،ANSI/ISOوالتي تُطبّقها جميع مُجمّعات + C+الحديثة. يعكس الكود في هذا الكتاب أسلوب وممارسات الترميز المعاصرة كما تُشجّعها لغة C++القياسية. مع ذلك، إذا كنت تستخدم مُجمّعًا أقدم، فقد لا يقبل جميع البرامج في هذا الكتاب. إليك السبب: خلال عملية التقييس، أضافت لجنة ANSI/ISOالعديد من الميزات الجديدة إلى اللغة.

عند تعريف هذه الميزات، نفّذها مطورو المجمّعات. وبالطبع، هناك دائمًا فترة زمنية بين إضافة ميزة جديدة إلى اللغة وتوفّرها في المجمّعات التجارية. وبما أن الميزات أُضيفت إلى لغة C++على مدى سنوات، فقد لا يدعم المجمّع القديم واحدةً أو أكثر منها. هذا مهم لأن إضافتين حديثتين إلى لغة C++تؤثران على كل برنامج ستكتبه، حتى أبسطها. إذا كنت تستخدم مجمّعًا قديمًا لا يقبل هذه الميزات الجديدة، فلا تقلق. هناك حل بديل سهل، وهو موضح هنا.

تتمثل الاختلافات الرئيسية بين الكود القديم والحديث في ميزتين: رؤوس الكود الجديدة وعبارة مساحة الاسم . لفهم هذه الاختلافات، سنبدأ باستعراض نسختين من برنامج C++بسيط وغير مُفعّل. يعكس الإصدار الأول المعروض هنا كيفية كتابة برامج C++باستخدام الكود القديم.

```
/*
برنامج C++على الطراز القديم.
*/

#تضمين <iostream.h>

int main() {

    العودة 0؛
}
```

انتبه جيدًا لعبارة . #includeفهي تتضمن الملف ،iostream.hوليس العنوان .<iostream>لاحظ أيضًا عدم وجود عبارة مساحة اسم .

Machine Translated by Google

وهنا النسخة الثانية من الهيكل العظمي، والتي تستخدم النمط الحديث.

```
/*
        برنامج C++بأسلوب حديث يستخدم الرؤوس ذات الأسلوب الجديد ومساحة الأسماء.

                */
        #تضمين <iostream>
        استخدام مساحة الاسم std؛

int main() {

        العودة 0؛
        }
```

يستخدم هذا الإصدار رأسًا بنمط C++ويحدد مساحة اسم. ذُكرت هاتان الميزتان سابقًا. لنلقِ نظرة فاحصة الآن.

عناوين C++الجديدة

كما تعلم، عند استخدام دالة مكتبة في برنامج، يجب تضمين رأسها. يتم ذلك باستخدام عبارة . include#على سبيل المثال، في لغة C، لتضمين رأس دوال الإدخال/الإخراج، يجب تضمين stdio.hبعبارة كالتالي:

```
#تضمين <stdio.h>
```

هنا، stdio.hهو اسم الملف الذي تستخدمه دوال الإدخال/الإخراج، والعبارة السابقة تُؤدي إلى تضمين هذا الملف في برنامجك. النقطة الأساسية هي أن عبارة include#هذه عادةً ما تتضمن ملفًا.

عندما تم اختراع لغة C++لأول مرة ولعدة سنوات بعد ذلك، استخدمت نفس الأسلوب من الرؤوس كما فعلت لغة .Cأي أنها استخدمت ملفات الرؤوس. في الواقع، لا تزال لغة C++القياسية تدعم الرؤوس بنمط C لملفات الرؤوس التي تُنشئها وللتوافق مع الإصدارات السابقة.

مع ذلك، أنشأت لغة C++القياسية نوعًا جديدًا من الرؤوس التي تستخدمها مكتبة C++القياسية. لا تحدد الرؤوس الجديدة أسماء الملفات، بل تحدد ببساطة مُعرِّفات قياسية يمكن للمُجمِّع ربطها بالملفات، مع أنها ليست ضرورية. تُعد رؤوس C++الجديدة تجريدًا يضمن ببساطة الإعلان عن النماذج الأولية والتعريفات المناسبة التي تتطلبها مكتبة .C++

بما أن العناوين الجديدة ليست أسماء ملفات، فهي لا تحمل امتداد .h . وتتكون فقط من اسم العنوان بين قوسين. على سبيل المثال، إليك بعض العناوين الجديدة التي يدعمها معيار .C++

<iostream> <fstream> <vector> <string>

Machine Translated by Google

يتم تضمين العناوين الجديدة باستخدام عبارة #include . الفرق الوحيد هو أن العناوين الجديدة لا تمثل بالضرورة أسماء الملفات.

لأن لغة ++C تتضمن مكتبة دوال C كاملةً، فإنها لا تزال تدعم ملفات الرؤوس القياسية المرتبطة بها. أي أن ملفات الرؤوس مثل stdio.h أو ctype.h لا تزال متاحة. مع ذلك، تُعرّف لغة ++C القياسية أيضًا رؤوسًا جديدة يمكنك استخدامها بدلًا من هذه الرؤوس. تُضيف إصدارات ++C من رؤوس C القياسية بادئة "c" إلى اسم الملف وتُحذف .h. على سبيل المثال، رأس الملف math.h الجديد في لغة ++C هو <cmath>، ورأس الملف string.h هو <cstring>. على الرغم من أنه يُسمح حاليًا بتضمين ملف رأس بنمط C عند استخدام دوال مكتبة C، إلا أن هذا النهج مُهمَل في لغة ++C القياسية (أي أنه غير مُوصى به). لهذا السبب، سيستخدم هذا الكتاب، من الآن فصاعدًا، رؤوس ++C الجديدة في جميع عبارات #include . إذا كان مُجمّعك لا يدعم الرؤوس الجديدة لمكتبة دوال C، فاستبدلها ببساطة بالرؤوس القديمة الشبيهة بلغة C.

بما أن الرأس الجديد إضافة حديثة نسبيًا إلى لغة C++، فستجد العديد من البرامج القديمة التي لا تستخدمه. تستخدم هذه البرامج رؤوسًا بنمط C، حيث يُحدد اسم الملف. وكما هو موضح في البرنامج الهيكلي القديم، فإن الطريقة التقليدية لإدراج رأس الإدخال/الإخراج هي كما هو موضح هنا.

```
#تضمين <iostream.h>
```

يؤدي هذا إلى تضمين الملف iostream.h في برنامجك. عادةً، يستخدم ملف الرأس القديم نفس اسم ملف الرأس الجديد المقابل له، مع إضافة .h .

حتى كتابة هذه السطور، تدعم جميع مُجمِّعات ++C رؤوس البرامج القديمة. مع ذلك، أُعلِن أن رؤوس البرامج القديمة قد أصبحت قديمة، ولا يُنصح باستخدامها في البرامج الجديدة. ولذلك، لم تُستخدَم في هذا الكتاب.

Remember على الرغم من أنها لا تزال شائعة في كود ++C الحالي، إلا أن الرؤوس ذات الطراز القديم أصبحت قديمة

عند تضمين رأس جديد في برنامجك، يُحفظ محتوى هذا الرأس في مساحة **الاسم** القياسية . مساحة الاسم هي ببساطة منطقة تعريفية. الغرض منها هو تحديد أسماء المعرفات لتجنب تضارب الأسماء.

العناصر المعلنة في مساحة اسم واحدة منفصلة عن العناصر المعلنة في مساحة اسم أخرى.

في الأصل، كانت أسماء دوال مكتبة C++، وما إلى ذلك، تُوضع ببساطة في مساحة الأسماء العامة (كما هو الحال في لغة C). ومع ذلك، مع ظهور العناوين الجديدة، وُضعت محتويات هذه العناوين في مساحة **الأسماء** القياسية . سنتناول مساحات الأسماء بمزيد من التفصيل لاحقًا في هذا الكتاب. أما الآن، فلن تحتاج للقلق بشأنها لأن العبارة

```
استخدام مساحة الاسم std؛
```

Machine Translated by Google

يؤدي إلى ظهور مساحة اسم std(أي أنه يضع stdفي مساحة الاسم العالمية).
بعد تجميع هذا البيان، لن يكون هناك فرق بين العمل مع رأس النمط القديم وآخر النمط الجديد.

نقطة أخرى: لضمان التوافق، عندما يتضمن برنامج C++رأس Cمثل stdio.h، يُضاف محتواه إلى مساحة الاسم العامة. هذا يسمح لمُجمِّع C++بتجميع برامج C-subset.

العمل مع مُجمِّع قديم: كما هو موضح، تُعدّ مساحات الأسماء والرؤوس الجديدة إضافات حديثة نسبيًا إلى لغة C++، أُضيفت أثناء عملية التوحيد. في حين أن جميع مُجمِّعات C++الجديدة تدعم هذه الميزات، فقد لا تدعمها المُجمِّعات القديمة. في هذه الحالة، سيُبلغ مُجمِّعك عن خطأ واحد أو أكثر عند محاولة ترجمة أول سطرين من برامج الأمثلة في هذا الكتاب. في هذه الحالة، هناك حل بديل سهل: ببساطة، استخدم رأسًا قديم الطراز واحذف عبارة مساحة الأسماء . أي، استبدل فقط

```
#تضمين <iostream>
استخدام مساحة الاسم std؛
```

مع

```
#تضمين <iostream.h>
```

يُحوّل هذا التغيير برنامجًا حديثًا إلى برنامج تقليدي. بما أن رأس البرنامج التقليدي يقرأ جميع محتوياته في مساحة الأسماء العالمية، فلا حاجة إلى جملة اسم .

نقطة أخرى: في الوقت الحالي، وفي السنوات القليلة القادمة، سترى العديد من برامج C++تستخدم رؤوسًا قديمة الطراز ولا تتضمن عبارة استخدام . سيتمكن مُجمِّع C++الخاص بك من تجميعها بكفاءة. مع ذلك، بالنسبة للبرامج الجديدة، يُنصح باستخدام النمط الحديث لأنه النمط الوحيد الذي يتوافق مع معيار .C++مع أن البرامج القديمة ستظل مدعومة لسنوات عديدة، إلا أنها غير متوافقة تقنيًا.

مقدمة عن فئات .C++يُقدّم هذا القسم أهم ميزة في :C++الفئة. لإنشاء كائن في C++، يجب عليك أولًا تعريف شكله العام باستخدام الكلمة المفتاحية .classتُشبه الفئة في تركيبها بنيةً لغويًا. إليك مثال: تُعرّف الفئة التالية نوعًا يُسمى stack، والذي سيتم استخدامه لإنشاء مكدس.

```
#تعريف الحجم 100
```

Machine Translated by Google


```
class stack { int stck[SIZE]; int tos; //يؤدي هذا إلى إنشاء مكدس الفئة.
public:
  void init(); void push(int i); int pop(); //عام:
};
```


قد تحتوي الفئة على أجزاء خاصة وعامة. جميع العناصر المُعرّفة في الفئة تكون خاصة افتراضيًا. على سبيل المثال، المتغيران stck وtos خاصان. هذا يعني أنه لا يمكن الوصول إليهما بواسطة أي دالة ليست عضوًا في الفئة. هذه إحدى طرق تحقيق التغليف، حيث يمكن التحكم بدقة في الوصول إلى عناصر بيانات معينة من خلال الحفاظ على خصوصيتها. على الرغم من عدم توضيح ذلك في هذا المثال، يمكنك أيضًا تعريف دوال خاصة، والتي لا يمكن استدعاؤها إلا بواسطة أعضاء آخرين في الفئة .

لجعل أجزاء من الفصل عامة (أي يمكن الوصول إليها بواسطة أجزاء أخرى من برنامجك)، يجب عليك تعريفها بعد الكلمة المفتاحية public . يمكن الوصول إلى جميع المتغيرات أو الدوال المُعرّفة بعد public بواسطة جميع الدوال الأخرى في البرنامج. باختصار، يصل باقي البرنامج إلى الكائن من خلال دواله public. مع أنه يمكنك استخدام متغيرات public، إلا أن الممارسة الجيدة تُملي عليك محاولة الحد من استخدامها.

بدلاً من ذلك، يجب عليك جعل جميع البيانات خاصة والتحكم في الوصول إليها عبر دوال عامة. ملاحظة أخرى: لاحظ أن الكلمة المفتاحية العامة تتبعها علامة نقطتين.
تُسمى الدوال push()، وpop()، وinit()، وstck، وtos فيُطلق عليهما اسم متغيرات أعضاء (أو عناصر بيانات). تذكر أن الكائن يُشكل رابطًا بين الكود والبيانات. الدوال الأعضاء فقط هي التي يمكنها الوصول إلى العناصر الخاصة بفئتها. وبالتالي، فإن push()، وpop()، وinit()، هي فقط من يمكنها الوصول إلى stck وtos.

بعد تعريف فئة، يمكنك إنشاء كائن من هذا النوع باستخدام اسم الفئة. باختصار، يصبح اسم الفئة مُحددًا جديدًا لنوع البيانات. على سبيل المثال، يُنشئ هذا كائنًا يُسمى mystack من نوع stack:

```
stack mystack;
```

عند إعلان كائن من فئة، فإنك تُنشئ مثيلًا لها. في هذه الحالة، mystack هو مثيل لـ stack. يمكنك أيضًا إنشاء كائنات عند تعريف الفئة بوضع أسمائها بعد أقواس الإغلاق، تمامًا كما تفعل مع الهياكل.

للمراجعة: في لغة C++، تُنشئ الفئة نوع بيانات جديد يُمكن استخدامه لإنشاء كائنات من هذا النوع. لذلك، يُعد الكائن مثيلًا للفئة، تمامًا كما يُعد متغير آخر مثيلًا لنوع البيانات int، على سبيل المثال. بعبارة أخرى، الفئة هي

Machine Translated by Google

التجريد المنطقي، في حين أن الكائن حقيقي. (أي أن الكائن موجود داخل ذاكرة الكمبيوتر.)

الشكل العام لإعلان الفئة البسيطة هو

الفئة class-name{ بيانات ووظائف
خاصة :publicبيانات ووظائف
عامة } اسم الكائن ؛list

بالطبع، قد تكون قائمة أسماء الكائنات فارغة.
داخل إعلان المكدس، تم تحديد الدوال الأعضاء باستخدام نماذجها الأولية. في لغة C++، يجب إنشاء نماذج أولية لجميع الدوال. النماذج الأولية ليست اختيارية.
يعمل النموذج الأولي لوظيفة عضو ضمن تعريف فئة بمثابة النموذج الأولي لتلك الوظيفة بشكل عام.

عند كتابة دالة تابعة لفئة، يجب عليك إخبار المُجمِّع بالفئة التي تنتمي إليها الدالة، وذلك بربط اسمها باسم الفئة التي تنتمي إليها. على سبيل المثال، إليك إحدى طرق كتابة دالة : ( )push

```
void stack::push(int i) {

إذا (tos == الحجم) {
cout << "المكدس ممتلئ.\n";
يعود؛
}
stck[tos] = i؛

هم+؛+
}
```

يُسمى :: عامل حل النطاق . يُخبر المُجمِّع أن هذا
إصدار ( )pushينتمي إلى فئة المكدس ، أو بمعنى آخر، أن ( )pushهذا يقع ضمن نطاق المكدس . في لغة C++، يمكن لعدة فئات مختلفة استخدام اسم الدالة نفسه. يعرف المُجمِّع أي دالة تنتمي إلى أي فئة بفضل مُعامل حل النطاق.

عند الإشارة إلى عنصر من فئة من خلال جزء من شيفرة غير تابعة لها، يجب عليك دائمًا القيام بذلك بالتزامن مع كائن من تلك الفئة. للقيام بذلك، استخدم اسم الكائن، متبوعًا بعامل النقطة، ثم اسم العنصر.

تنطبق هذه القاعدة سواءً كنتَ تصل إلى عنصر بيانات أو عنصر دالة. على سبيل المثال، تستدعي هذه القاعدة ( )initلكائن .stack1

```
المكدس المكدس ،1المكدس ؛2

stack1.init();
```

Machine Translated by Google

يؤدي هذا الجزء إلى إنشاء كائنين، stack1 وstack2، ويقوم بتهيئة stack1.
افهم أن stack1 وstack2 كائنان منفصلان. هذا يعني، على سبيل المثال، أن تهيئة stack1 لا تؤدي إلى تهيئة stack2 أيضًا. العلاقة الوحيدة بين stack1 وstack2 هي أنهما كائنان من نفس النوع.

داخل الفصل، يمكن لوظيفة عضو واحدة استدعاء وظيفة عضو أخرى أو الإشارة إلى
عنصر بيانات مباشرةً، دون استخدام مُعامل النقطة. فقط عند الإشارة إلى عنصر بواسطة رمز لا ينتمي إلى الفئة، يجب استخدام اسم الكائن ومُعامل النقطة.

يقوم البرنامج الموضح هنا بتجميع كل القطع والتفاصيل المفقودة
يوضح فئة المكدس :


```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

#تعريف الحجم 100

class stack { int stck[SIZE]; int tos; //يؤدي هذا إلى إنشاء مكدس الفئة.



void init(); void push(int i); int pop(); :عام



};

void stack::init() {

هم ؛0 =
}

void stack::push(int i) {

إذا (tos == الحجم) {
cout << "المكدس ممتلئ.";\n
يعود؛
} stck[tos] = i؛

هم++؛
}

int stack::pop() {

إذا (tos == 0) {
```

Machine Translated by Google


```
                \n"; return 0;.تدفق المكدس"cout <<

                }
                أولئك--؛
            stck[tos]؛ العودة
                }

int main() {

                                    stack stack1، stack2؛ // إنشاء كائنين مكدسين

stack1.init(); stack2.init();


        stack2.push(2);
        stack1.push(1);

        stack2.push(4);
        stack1.push(3);

cout << stack2.pop() << cout << stack2.pop() << "\n";
    cout << stack1.pop() << cout << stack1.pop() <<
                                    " ";


        0؛ العودة
            }
```

يظهر الناتج من هذا البرنامج هنا.

```
3 1 4 2
```

نقطة أخيرة: تذكر أن الأعضاء الخاصة للكائن لا يمكن الوصول إليها إلا من خلال الدوال التي هي أعضاء في هذا الكائن. على سبيل المثال، عبارة مثل

```
                                    stack1.tos = 0; // خطأ، tos خاص.
```

لا يمكن أن يكون في **الوظيفة** الرئيسية للبرنامج السابق لأن tos خاصة.

Machine Translated by Google

التحميل الزائد للوظائف

إحدى الطرق التي يحقق بها C++تعدد الأشكال هي من خلال استخدام التحميل الزائد للوظيفة.
في لغة C++،يمكن لدالتين أو أكثر أن تتشاركا الاسم نفسه طالما أن تعريفات معاملاتهما مختلفة. في هذه الحالة، تُسمى الدوال التي تتشارك الاسم نفسه "مُحمَّلة بشكل زائد"، وتُعرف هذه العملية باسم "التحميل الزائد للدالة".

لمعرفة سبب أهمية التحميل الزائد للوظائف، ضع في اعتبارك أولاً ثلاث وظائف محددة
باستخدام مجموعة Cالفرعية: abs()، labs()وfabs(). تُرجع الدالة abs()القيمة المطلقة لعدد صحيح، وتُرجع labs()القيمة المطلقة لقيمة طويلة، وتُرجع fabs()القيمة المطلقة لقيمة مزدوجة . مع أن هذه الدوال تُؤدي عمليات متطابقة تقريبًا، إلا أنه في لغة C++،يجب استخدام ثلاثة أسماء مختلفة قليلاً لتمثيل هذه المهام المتشابهة أساسًا. هذا يجعل الوضع أكثر تعقيدًا من الناحية النظرية. فرغم أن المفهوم الأساسي لكل دالة هو نفسه، إلا أنه يجب على المبرمج تذكر ثلاثة أشياء، وليس شيئًا واحدًا فقط. مع ذلك، في لغة C++،يمكنك استخدام اسم واحد فقط لجميع الدوال الثلاث، كما يوضح هذا البرنامج:


```
#تضمين <iostream>
استخدام مساحة الاسم std؛

//يتم تحميل absبثلاث طرق ;(long l) abs ;(double d) abs
int abs(int i); double

int main() {
    cout << abs(-10) << "\n";
    cout << abs(-11.0) << "\n";
    cout << abs(-9L) << "\n";
    العودة 0؛
}

int abs(int i) {
    cout << "استخدام عدد صحيح ;()abs\n";
```

Machine Translated by Google


```
العودة ؛i : i- ؟0>i
}

{ (مزدوج d) مزدوجabs

"cout << "استخدام abs()\n";

العودة ؛d : d- ؟0.0>d
}

{ (طويل) sba طويل

"cout << "استخدام abs()\nالطويل";

العودة ؛l : l- ؟0>l
}
```

يظهر الناتج من هذا البرنامج هنا.

```
استخدام عدد صحيح abs()
10
استخدام abs()المزدوج
11
استخدام abs()الطويل
9
```

يُنشئ هذا البرنامج ثلاث دوال متشابهة، وإن كانت مختلفة، تُسمى abs()، وكل منها تُرجع القيمة المطلقة لمُعاملها. يعرف المُترجم أي دالة يُستدعى في كل حالة بناءً على نوع المُعامل. تكمن فائدة الدوال المُحمّلة في أنها تسمح بالوصول إلى مجموعات دوال مُرتبطة باسم مُشترك. وبالتالي، يُمثل اسم abs() الإجراء العام المُنفذ. ويُترك للمُترجم اختيار الطريقة المُحددة المُناسبة لكل حالة.

ما عليك سوى تذكر الإجراء العام المُنفَّذ. بفضل تعدد الأشكال، تم اختصار ثلاثة أمور يجب تذكرها إلى أمر واحد. هذا المثال بسيط نسبيًا، ولكن إذا توسعت في المفهوم، يمكنك أن ترى كيف يُمكن لتعدد الأشكال أن يُساعدك في إدارة برامج مُعقَّدة للغاية.

بشكل عام، لزيادة تحميل دالة، يكفي إعلان إصدارات مختلفة منها. يتولى المُجمِّع الباقي. يجب مراعاة شرط مهم عند زيادة تحميل دالة: يجب أن يختلف نوع و/أو عدد معاملات كل دالة مُحمَّلة. لا يكفي أن تختلف دالتان في أنواع إرجاعهما فقط، بل يجب أن تختلفا في أنواع أو عدد معاملاتهما. (لا تُوفِّر أنواع الإرجاع معلومات كافية في جميع الحالات للمُجمِّع لتحديد الدالة التي سيستخدمها). بالطبع، قد تختلف الدوال المُحمَّلة أيضًا في أنواع إرجاعها.

Machine Translated by Google

وهنا مثال آخر يستخدم وظائف زائدة عن الحد:

```
#تضمين <iostream>
#تضمين <cstdio>
#include <cstring>باستخدام مساحة الاسم std؛

stradd(char *s1، int i)؛ باطلstradd(char *s1، char *s2)؛ باطل

int main() {

char str[80]؛

strcpy(str, "مرحبا"); stradd(str, "هناك");
cout << str << "\n";

stradd(str، 100)؛ cout << str << "\n"؛

العودة 0؛
}

void stradd(char *s1, char *s2) { //ربط سلسلتين

strcat(s1, s2);
}

void stradd(char *s1, int i) { //قم بربط سلسلة مع عدد صحيح "مُحوّل إلى سلسلة"

char temp[80]؛

sprintf(temp, "%d", i); strcat(s1, temp);

}
```

في هذا البرنامج، تُحمَّل دالة stradd()بشكل زائد. أحد الإصدارين يجمع سلسلتين نصيتين (تمامًا كما تفعل strcat() .) الإصدار الآخر يُحوِّل عددًا صحيحًا إلى سلسلة نصية، ثم يُضيفه إلى سلسلة نصية. هنا، يُستخدم التحميل الزائد لإنشاء واجهة تُضيف إما سلسلة نصية أو عددًا صحيحًا إلى سلسلة نصية أخرى.

Machine Translated by Google

يمكنك استخدام الاسم نفسه لزيادة تحميل الدوال غير المرتبطة، ولكن لا ينبغي عليك ذلك. على سبيل المثال، يمكنك استخدام الاسم sqr( ) لإنشاء دوال تُرجع مربع عدد صحيح وجذر عدد مزدوج . مع ذلك، تختلف هاتان العمليتان اختلافًا جوهريًا؛ فتطبيق زيادة تحميل الدوال بهذه الطريقة يُفسد غرضه (بل يُعتبر في الواقع أسلوب برمجة خاطئ). عمليًا، يجب زيادة تحميل العمليات المرتبطة ارتباطًا وثيقًا فقط.

التحميل الزائد للمشغل

يتم تحقيق تعدد الأشكال في لغة C++ أيضًا من خلال التحميل الزائد للمشغلات. كما تعلم، في لغة C++، من الممكن استخدام المشغلين >> و << لإجراء عمليات الإدخال/الإخراج على وحدة التحكم.
يمكنهم إجراء هذه العمليات الإضافية لأن هذه العوامل مُحمّلة بشكل زائد في رأس <iostream> . عند تحميل عامل بشكل زائد، فإنه يأخذ معنى إضافيًا بالنسبة لفئة معينة. ومع ذلك، فإنه لا يزال يحتفظ بجميع معانيه القديمة.

بشكل عام، يمكنك زيادة تحميل معظم معاملات C++ بتعريف معناها بالنسبة لفئة محددة. على سبيل المثال، تذكر فئة المكدس التي طُوّرت سابقًا في هذا الفصل. من الممكن زيادة تحميل عامل + بالنسبة لكائنات من نوع stack بحيث يُضيف محتويات مكدس إلى محتويات آخر. مع ذلك، يحتفظ عامل + بمعناه الأصلي بالنسبة لأنواع البيانات الأخرى.

نظرًا لأن التحميل الزائد للمشغل، في الممارسة العملية، أكثر تعقيدًا إلى حد ما من التحميل الزائد للوظيفة التحميل الزائد، يتم تأجيل الأمثلة إلى الفصل 14.

الوراثة: كما ذُكر سابقًا في هذا الفصل، تُعد الوراثة إحدى السمات الرئيسية للغات البرمجة كائنية التوجه. في لغة C++، يُدعم الوراثة بالسماح لفئة ما بدمج فئة أخرى في إعلانها. تتيح الوراثة بناء تسلسل هرمي للفئات، بدءًا من الأكثر عموميةً إلى الأكثر تحديدًا. تتضمن العملية أولًا تعريف فئة أساسية، والتي تُحدد الصفات المشتركة بين جميع الكائنات المراد اشتقاقها من الفئة الأساسية. تُمثل الفئة الأساسية الوصف الأكثر عمومية. عادةً ما تُسمى الفئات المشتقة من الفئة الأساسية بالفئات المشتقة. تتضمن الفئة المشتقة جميع ميزات الفئة الأساسية العامة، ثم تُضيف صفات خاصة بها. لتوضيح كيفية عمل ذلك، يُنشئ المثال التالي فئات تُصنف أنواعًا مختلفة من المباني.

للبدء، يتم إعلان فئة البناء ، كما هو موضح هنا. ستكون بمثابة الأساس لـ فئتين مشتقتين.

مبنى الفصل {غرف داخلية؛
طوابق داخلية؛ مساحة داخلية؛

Machine Translated by Google


```
        void set_area(int num); int get_area(); عام:
        void set_floors(int num); int get_floors();
        void set_rooms(int num); int get_rooms();




};
```

لأن جميع المباني (على سبيل المثال) تشترك في ثلاث خصائص: غرفة واحدة أو أكثر، طابق واحد أو أكثر، ومساحة إجمالية، فإن فئة المبنى تُجسّد هذه المكونات في إعلانها. الدوال الأعضاء التي تبدأ بـ setتُعيّن قيم البيانات الخاصة. الدوال التي تبدأ بـ get تُرجع هذه القيم.

يمكنك الآن استخدام هذا التعريف الشامل للمبنى لإنشاء فئات مشتقة تصف أنواعًا محددة من المباني. على سبيل المثال، إليك فئة مشتقة تُسمى "منزل":

```
//المنزل مشتق من فئة المبنى : houseمبنى عام {

    intغرف نوم؛ intحمامات؛ عام: int get_baths(); num);
    void set_bedrooms(int num); int get_bedrooms(); void set_baths(int




};
```

لاحظ كيف يُورث البناء . الشكل العام للوراثة هو

```
                الفئة المشتقة من الفئة: الوصول إلى الفئة
                الأساسية // }نص الفئة الجديدة
}
```

الوصول هنا اختياري. ومع ذلك، إن وُجد، يجب أن يكون عامًا أو خاصًا أو محميًا.
(سيتم شرح هذه الخيارات بمزيد من التفصيل في الفصل .(١٢في الوقت الحالي، ستستخدم جميع الفئات الموروثة .publicاستخدام publicيعني أن جميع العناصر publicللفئة الأساسية ستصبح أعضاءً publicللفئة المشتقة. وبالتالي، تصبح العناصر publicللفئة buildingأعضاءً publicللفئة المشتقة ،houseوتكون متاحة لدوال الأعضاء في houseكما لو كانت معلنة داخل .houseمع ذلك، لا تتمتع دوال الأعضاء في houseبإمكانية الوصول إلى العناصر الخاصة في .buildingهذه نقطة مهمة. على الرغم من أن house يرث ،buildingإلا أنه يتمتع بإمكانية الوصول فقط إلى

Machine Translated by Google

أعضاء المبنى العامون . بهذه الطريقة، لا يتعارض الوراثة مع مبادئ التغليف اللازمة للبرمجة الكائنية التوجه.

Remember

تتمتع الفئة المشتقة بإمكانية الوصول المباشر إلى أعضائها والأعضاء العامة للفئة الأساسية.

هذا برنامج يوضح الوراثة. يُنشئ هذا البرنامج فئتين مشتقتين من المباني باستخدام الوراثة؛ إحداهما منزل، والأخرى مدرسة.

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

مبنى الفصل int }الغرف؛ intالطوابق؛ intالمساحة؛ عام:
int get_floors(); void set_area(int num)؛ int get_area();
num)؛ int get_rooms(); void set_floors(int num)؛
void set_rooms(int

};

//يتم اشتقاق المنزل من فئة المبنى : houseمبنى عام int }غرف نوم؛ int
get_bedrooms(); void set_baths(int num); int get_baths(); عام: حمامات؛
void set_bedrooms(int num); int

};

//المدرسة مشتقة أيضًا من فئة المبنى :schoolمبنى عام {

الفصول الدراسية ؛intالمكاتب ؛intالعامة: ؛(set_classrooms(int num
void
```

Machine Translated by Google


```
        void set_offices(int num); int get_offices();
                                int get_classrooms();

};

                                num) { tni)مجموعة_غرف::مبنى فارغ

        الغرف =عدد؛
                }

                                num) { tni)مجموعة_الطوابق::مبنى فارغ

        الطوابق =العدد؛
                }

                            num) { tni)مجموعة_المنطقة::بناء باطل

    المساحة =العدد؛
                }

            int building::get_rooms() {

        غرف العودة؛
                }

                int building::get_floors() {

        أرضيات العودة؛
                }

            int building::get_area() {

        منطقة العودة؛
                }

                            num) { tni)مجموعة غرف النوم::منزل فارغ

        غرف النوم =العدد؛
                }

                void house::set_baths(int num) {
```

Machine Translated by Google


```
الحمامات =رقم؛
}

int house::get_bedrooms() {

غرف نوم العودة؛
}

int house::get_baths() {

حمامات العودة؛
}

num) { tni)smoorssalc_tes::مدرسة فارغة

الفصول الدراسية =العدد؛
}

num) { tni)مدرسة فارغة::مجموعة_المكاتب

المكاتب =رقم؛
}

int school::get_classrooms() {

عودة الفصول الدراسية؛
}

int school::get_offices() {

مكاتب العودة؛
}

int main() {

البيت ح؛ المدرسة
س؛

h.set_bedrooms(5);
h.set_floors(3); h.set_area(4500);
h.set_rooms(12);
```

Machine Translated by Google


```
h.set_baths(3);

" البيت لديه"cout <<                    ;"n\غرف النوم<< h.get_bedrooms(); "
كوت <<

s.set_offices(5); s.set_area(25000);
s.set_rooms(200); s.set_classrooms(180);

                                                                    ;"n\ن"
                                                                    ;"مساحتها هي "cout
                                                                    ;s.get_area();

العودة 0؛
}
```

يظهر هنا الناتج الذي أنتجه هذا البرنامج.

```
المنزل يحتوي على 5غرف نوم
المدرسة لديها 180فصلا دراسيا
مساحتها 25000
```

كما يوضح هذا البرنامج، فإن الميزة الرئيسية للوراثة هي إمكانية إنشاء تصنيف عام يمكن دمجه في تصنيفات أكثر تحديدًا. بهذه الطريقة، يمكن لكل كائن تمثيل فئته الفرعية بدقة.

عند الكتابة عن ،++Cيتم استخدام المصطلحين الأساسي والمشتق بشكل عام لوصف علاقة الوراثة. ومع ذلك، يُستخدم أيضًا مصطلحا "الوالد" و "الطفل" . قد تجد أيضًا مصطلحي "الطبقة العليا" و "الطبقة الفرعية".

إلى جانب توفير مزايا التصنيف الهرمي، يُتيح الوراثة أيضًا دعمًا لتعدد أشكال التشغيل من خلال آلية الدوال الافتراضية . (راجع الفصل السادس عشر للمزيد من التفاصيل).

من الشائع جدًا أن يتطلب جزء من الكائن تهيئةً قبل استخدامه. على سبيل المثال، تذكر فئة المكدس التي طُوّرت سابقًا في هذا الفصل.

قبل استخدام المكدس، كان يجب ضبط قيمة tosإلى صفر. وقد تم ذلك باستخدام الدالة . ()initولأن متطلبات التهيئة شائعة جدًا، تسمح لغة ++Cللكائنات بتهيئة نفسها عند إنشائها. ويتم هذا التهيئة التلقائية باستخدام دالة منشئ.

Machine Translated by Google

المُنشئ هو دالة خاصة تابعة لفئة، وتحمل نفس اسم تلك الفئة. على سبيل المثال، هكذا تبدو فئة المكدس عند تحويلها لاستخدام مُنشئ للتهيئة:

```
class stack { int stck[SIZE]; int tos; //يؤدي هذا إلى إنشاء مكدس الفئة.

public: stack(); // منشئ void push(int i); int pop();

};
```

لاحظ أن مُنشئ ( )stack لا يُحدد نوع إرجاع. في C++، لا يُمكن للمُنشئات إرجاع قيم، وبالتالي لا تحتوي على نوع إرجاع.

يتم ترميز المنشئ ( )stack على النحو التالي :

```
stack::stack() { //منشئ المكدس

tos = 0; cout << "تم تهيئة المكدس\n";

}
```

تذكّر أن رسالة "تم تهيئة المكدس" تُخرَج لتوضيح المُنشئ. في الواقع، لا تُخرِج معظم المُنشئات أي شيء.

سيقومون ببساطة بإجراء عمليات تهيئة مختلفة.
يُستدعى مُنشئ الكائن تلقائيًا عند إنشائه. هذا يعني أنه يُستدعى عند تنفيذ إعلان الكائن. إذا كنتَ معتادًا على اعتبار عبارة الإعلان سلبية، فهذا ليس الحال في لغة C++، ففي C++، عبارة الإعلان هي عبارة تُنفَّذ. هذا التمييز ليس أكاديميًا فحسب، بل قد يكون للكود المُنفَّذ لإنشاء كائن أهمية بالغة. يُستدعى مُنشئ الكائن مرة واحدة للكائنات العامة أو المحلية الثابتة . أما بالنسبة للكائنات المحلية، فيُستدعى المُنشئ في كل مرة يتم فيها مواجهة إعلان الكائن.

مُكمِّل المُنشئ هو المُدمِّر. في كثير من الحالات، يحتاج الكائن إلى تنفيذ إجراء أو إجراءات عند تدميره. تُنشأ الكائنات المحلية عند إدخال كتلتها، وتُدمَّر عند تركها. تُدمَّر الكائنات العامة عند انتهاء البرنامج. عند تدمير كائن، يُستدعى مُدمِّره (إن وُجد) تلقائيًا. هناك العديد من الأسباب التي قد تستدعي استخدام المُدمِّر. على سبيل المثال، قد يحتاج الكائن إلى إلغاء تخصيص ذاكرة سبق تخصيصها، أو قد يحتاج إلى إغلاق ملف كان قد فتحه. في لغة C++، المُدمِّر هو الذي يُعالج أحداث التعطيل. يحمل المُدمِّر نفس اسم...

Machine Translated by Google

مُنشئ، ولكنه يسبقه .~على سبيل المثال، إليك فئة المكدس ومنشئها ومُدمرها. (لاحظ أن فئة المكدس لا تتطلب مُدمرًا؛ الموضح هنا هو للتوضيح فقط).

```
//يؤدي هذا إلى إنشاء مكدس الفئة. class stack { int stck[SIZE]; int tos;

public: stack(); // منشئ ~stack(); // مدمر void push(int i); int pop();

};

stack::stack() { //منشئ المكدس
  tos = 0; cout << "تم تهيئة المكدس\n";
}

stack::~stack() { //مدمر المكدس
  cout << "تم تدمير المكدس\n";
}
```

لاحظ أنه، مثل المنشئين، لا تحتوي المدمرات على قيم إرجاع.
لمعرفة كيفية عمل المنشئين والمدمرين، إليك إصدار جديد من المكدس
البرنامج الذي تمت مناقشته سابقًا في هذا الفصل. لاحظ أن ()initلم تعد ضرورية.

```
//استخدام المنشئ والمدمر.
#تضمين <iostream>
استخدام مساحة الاسم ؛std

#تعريف الحجم 100

class stack { int stck[SIZE]; int tos; //يؤدي هذا إلى إنشاء مكدس الفئة.

عام:
```

Machine Translated by Google


```
i); int pop(); المدمر ~stack(); // المنشئ stack();
                                void push(int

};

        stack::stack() { منشئ المكدس //

                        ;"n\تم تهيئة المكدس"tos = 0; cout <<

}

        stack::~stack() { مدمر المكدس //

                        ;"n\تم تدمير المكدس"cout <<
                                        }

                void stack::push(int i) {

                                        { (الحجم)tos == إذا
                                "n\المكدس ممتلئ.";cout <<
                                        يعود؛
                } stck[tos] = i؛

        هم+؛+
                }

                int stack::pop() {

                0؛ العودة \n"؛"تدفق المكدس. (tos == 0) { cout << إذا

                                }
                                أولئك--؛
                        stck[tos]؛ العودة
                                }

        int main() {

                                المكدس أ، ب؛ // إنشاء كائنين مكدسين
```

Machine Translated by Google


```
    أ.ادفع(1)؛
    ب.ادفع(2)؛

    أ.ادفع(3)؛
    ب.ادفع(4)؛

<< cout << b.pop() << cout << b.pop() <<"\n";
    cout << a.pop() << cout << a.pop()
                                " ";


    العودة ؛0
}
```


يعرض هذا البرنامج ما يلي:

```
تم تهيئة المكدس
تم تهيئة المكدس
3 1 4 2
تم تدمير المكدس
تم تدمير المكدس
```

كلمات مفتاحية C++

هناك 63كلمة رئيسية مُعرّفة حاليًا للغة C++القياسية. وهي موضحة في
الجدول .1-11مع بناء الجملة الرسمي للغة C++، تُشكل هذه اللغات برمجة .C++
اللغة. كما أن الإصدارات المبكرة من C++حددت كلمة overload ، ولكنها أصبحت قديمة.
ضع في اعتبارك أن لغة C++هي لغة حساسة لحالة الأحرف وتتطلب أن تكون جميع الكلمات الرئيسية
يكون بأحرف صغيرة.

استراحة	منطقي	آلي	اسم
فصل	حرف	يمسك	قضية
تقصير	يكمل	ثابت_صب	ثابت
البث الديناميكي	مزدوج	يفعل	يمسح
يصدّر	صريح	عدّ	آخر

الجدول 1-11: الكلمات الرئيسية في لغة C++

Machine Translated by Google

ل	يطفو	خطأ شنيع	خارجي
مضمن	لو	انتقل	صديق
مساحة الاسم	متقلب	إلى طويل	عدد صحيح
محمي	إعادة تفسير خاصة	سجل المشغل	جديد
يعود		موقّع	عام
ثابت	حجم		قصير
نموذج	يُحوّل	هيكل	البث الثابت
يحاول	حقيقي	يرمي	هذا
الاتحاد	اسم النوع	معرف	نوع تعريف غير
فارغ	افتراضي	النوع باستخدام	موقّع
	بينما	wchar_t	متقلب

الفصل 11: أساسيات لغة C++

الشكل العام لبرنامج C++

على الرغم من أن الأنماط الفردية تختلف، فإن معظم برامج C++ سيكون لها هذا الشكل العام:

```
#يتضمن
إعلانات الفئة الأساسية
إعلانات الفئة المشتقة
نماذج أولية للوظائف غير العضو
int main( )
{
//...
}
تعريفات الدوال غير العضويّة
```

في معظم المشاريع الكبيرة، سيتم وضع جميع إعلانات الفئة في ملف رأس وإدراجها
مع كل وحدة. لكن التنظيم العام للبرنامج يبقى كما هو.
وتتناول الفصول المتبقية في هذا القسم بمزيد من التفصيل الميزات
تمت مناقشتها في هذا الفصل، بالإضافة إلى جميع الجوانب الأخرى الخاصة بـ .C++

Machine Translated by Google

The Complete Reference



الفصل الثاني عشر

الفئات والكائنات

جميع الحقوق محفوظة ٢٠٠٣ ©لشركة ماكجرو هيل. انقر هنا للاطلاع على شروط الاستخدام.

Machine Translated by Google

في لغة C++، تُشكل الفئة أساس البرمجة الكائنية التوجه. تُستخدم الفئة

تُنشأ الفئات باستخدام الكلمة المفتاحية class . يُعرّف إعلان الفئة نوعًا جديدًا يربط بين الكود والبيانات. ويُستخدم هذا النوع الجديد بعد ذلك لإعلان كائنات تلك الفئة.

وبالتالي، فإن الفئة تجريد منطقي، أما الكائن فله وجود مادي. بمعنى آخر، الكائن هو مثال للفئة.

إعلان الفئة مشابه لبنية الجملة. في الفصل الحادي عشر، تم شرح مُبسط
تم عرض الشكل العام لإعلان الفئة. هذا هو الشكل العام الكامل لإعلان الفئة الذي لا يرث أي فئة أخرى.

الفئة class-name{ بيانات خاصة
ووظائف :access-specifier
بيانات ووظائف :access-specifier
بيانات ووظائف // ... :specifier
-accessبيانات ووظائف } قائمة
الكائنات؛

قائمة الكائنات اختيارية. إذا وُجدت، فهي تُعلن عن كائنات الفئة. هنا، مُحدِّد الوصول هو أحد الكلمات المفتاحية الثلاث التالية في لغة :C++

العامة والخاصة

المحمية

افتراضيًا، تكون الدوال والبيانات المُعلنة داخل فئة ما خاصة بتلك الفئة، ولا يمكن الوصول إليها إلا من قِبل أعضاء آخرين فيها. يسمح مُحدد الوصول العام بالوصول إلى الدوال أو البيانات من أجزاء أخرى من برنامجك. لا يلزم مُحدد الوصول المحمي إلا عند وجود وراثة (انظر الفصل .(15بمجرد استخدام مُحدد وصول، يبقى ساريًا حتى يتم العثور على مُحدد وصول آخر أو حتى انتهاء إعلان الفئة.

يمكنك تغيير مواصفات الوصول بقدر ما تريد ضمن إعلان الفصل .
على سبيل المثال، يمكنك التبديل إلى عام لبعض الإعلانات، ثم العودة إلى خاص . يوضح إعلان الفئة في المثال التالي هذه الميزة:

Machine Translated by Google


```
#تضمين <iostream>
<cstring> #include باستخدام مساحة الاسم
std؛

موظف الصف {
private: double wage; // هذه public: void putname(char *n); // خاص بشكل افتراضي char name[80];
public: void putwage(double w); // خاص مرة أخرى الآن، public void getname(char *n);
العودة إلى public double getwage(); };

void employee::putname(char *n) {
n)؛ ،الاسم)ypcrts
}

void employee::getname(char *n) {
؛(n، strcpy(الاسم
}

void employee::putwage(double w) {
الأجر =و؛
}

موظف مزدوج::الحصول على أجر { ()
أجر العائد؛
}

int main() {
الموظف تيد؛ اسم الشخصية[08]؛

ted.putwage(75000); ;("تيد جونز")emantup.det
```

Machine Translated by Google


```
                                                    emanteg.det(الاسم)؛
                                                    cout << name <<                    "يصنع >>
                                                                                                "
                                                                    $"; cout << ted.getwage()                كل سنة."؛

                                                    العودة ؛0
                                                    }
```

هنا، الموظف هو فئة بسيطة يتم استخدامها لتخزين اسم الموظف وأجره.
لاحظ أنه تم استخدام محدد الوصول العام مرتين.
مع أنه يمكنك استخدام مُحدِّدات الوصول متى شئت ضمن إعلان الفئة، إلا أن ميزتها الوحيدة هي أن التجميع البصري لأجزاء مختلفة من الفئة يُسهِّل على قارئ البرنامج فهمه. مع ذلك، بالنسبة للمُجمِّع، لا يُحدث استخدام مُحدِّدات وصول متعددة أي فرق.

في الواقع، يجد معظم المبرمجين أنه من الأسهل وجود قسم واحد فقط خاص، ومحمي، وعام داخل كل فئة. على سبيل المثال، يقوم معظم المبرمجين ببرمجة فئة الموظف كما هو موضح هنا، مع تجميع جميع العناصر الخاصة معًا وتجميع جميع العناصر العامة معًا:

```
فئة الموظف  ;[80]char name{أجر مضاعف؛ عام: ; void putname(char *n); void getname(char *n); void putwage(double w); double getwage(); };
```

الدوال المُعلنة داخل فئة تُسمى دوال أعضاء. تستطيع الدوال الأعضاء الوصول إلى أي عنصر من عناصر الفئة التي تكون جزءًا منها. وهذا يشمل جميع العناصر الخاصة .
تُسمى المتغيرات التي تُمثل عناصر فئة متغيرات أعضاء أو عناصر بيانات. (يُستخدم أيضًا مصطلح متغير المثيل). ويمكن الإشارة إلى أي عنصر من عناصر الفئة مجتمعًا بأنه عضو في تلك الفئة.

هناك بعض القيود التي تنطبق على أعضاء الفئة. لا يمكن لمتغير عضو غير ثابت أن يحتوي على مُهيئ. لا يمكن لأي عضو أن يكون كائنًا للفئة التي يتم إعلانها. (مع أن العضو يمكن أن يكون مؤشرًا للفئة التي يتم إعلانها). لا يمكن إعلان أي عضو كـ autoأو extern أو register.

بشكل عام، يجب جعل جميع عناصر بيانات فئة ما خاصة بتلك الفئة. هذا جزء من عملية التغليف. مع ذلك، قد تحتاج في بعض الحالات إلى جعل متغير واحد أو أكثر عامًا. (على سبيل المثال، متغير ذو أهمية كبيرة

Machine Translated by Google

قد يلزم أن يكون المتغير المستخدم متاحًا عالميًا لتحقيق أوقات تشغيل أسرع.)
عندما يكون المتغير عامًا، يُمكن الوصول إليه مباشرةً من أي جزء آخر من برنامجك. صيغة الوصول إلى عنصر بيانات عام هي نفسها صيغة استدعاء دالة عضو: حدد اسم الكائن، وعامل النقطة، واسم المتغير.

يوضح هذا البرنامج البسيط استخدام متغير عام:

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

الصف myclass{ عام:

// int i, j, k; يمكن الوصول إليها من خلال البرنامج بأكمله
};

int main() {

صفي أ، ب؛

منظمة العفو الدولية // ؛100 =الوصول إلى jو k على ما يرام ;4 = aj أك =منظمة العفو الدولية *
آج؛

// bk = 12; تذكر، bk وak مختلفان ;bk >>
cout << ak << " ";

العودة ؛0
}
```

الهياكل والفئات مترابطة. **الهياكل جزء من لغة** C، وقد ورثتها من هذه اللغة. كما رأيت، تُشبه الفئة نحويًا البنية. لكن العلاقة بين الفئة والبنية أقرب مما قد تظن للوهلة الأولى. في لغة C++، تم توسيع دور البنية، مما جعلها طريقة بديلة لتحديد الفئة. في الواقع، الفرق الوحيد بين الفئة والبنية هو أن جميع الأعضاء تكون افتراضيًا عامة في البنية وخاصة في الفئة. أما في الجوانب الأخرى، فالهياكل والفئات متكافئة. أي أن البنية في C++ تُعرّف نوع الفئة. **على سبيل المثال**، لننظر إلى هذا البرنامج القصير، الذي يستخدم بنية لإعلان فئة تتحكم في الوصول إلى سلسلة نصية:

```
//استخدام هيكل لتحديد فئة.
#تضمين <iostream>
#تضمين <cstring>
```

Machine Translated by Google


```
                استخدام مساحة الاسم std؛

mystr { void buildstr(char *s); // public void showstr(); private: //
                                        الآن انتقل إلى الوضع الخاص lstruct

                                                                char str[255]; } ;


                                                        void mystr::buildstr(char *s) {

                                        else strcat(str, s); التهيئة السلسلة if(!*s) *str = '\0'; //

}

                mystr::showstr() { باطل

                        cout << str << "\n";
                                }

    int main() {

        mystr s;

        "); مرحبا")rtsdliub.ss.buildstr(""); // init
                                ;("هناك!")rtsdliub.s


            s.showstr();

          0؛ العودة
                }
```

يعرض هذا البرنامج السلسلة "مرحبا!".
يمكن إعادة كتابة الفصل mystr باستخدام الفصل كما هو موضح هنا:

```
        mystr { الصف
void showstr(); } ؛ عام void buildstr(char *s)؛ // :عام char str[255]؛
```

Machine Translated by Google

ربما تتساءل لماذا يحتوي ++Cعلى الكلمتين الرئيسيتين المتماثلتين تقريبًا struct
والفئة . هذا التكرار الظاهري مُبرَّرٌ لعدة أسباب. أولًا، لا يوجد سبب جوهري لعدم زيادة إمكانيات البنية. في لغة ،Cتُوفِّر الهياكل بالفعل وسيلةً لتجميع البيانات. لذا، يُعدُّ السماح لها بتضمين دوال أعضاء خطوةً بسيطة. ثانيًا، نظرًا لترابط الهياكل والفئات، فقد يكون من الأسهل نقل برامج Cالحالية إلى .+ +Cأخيرًا، على الرغم من أن structو classمُتكافئان تقريبًا اليوم، فإن توفير كلمتين مفتاحيتين مختلفتين يسمح بتطور تعريف الفئة بحرية . لكي تظل ++Cمتوافقة مع ،Cيجب أن يكون تعريف structمرتبطًا دائمًا بتعريف Cالخاص بها.

على الرغم من أنه يمكنك استخدام هيكل حيث تستخدم فئة، إلا أن معظم المبرمجين لا يفعلون ذلك.
عادةً ما يُفضّل استخدام فئة عند الحاجة إلى فئة، وبنية عند الحاجة إلى بنية شبيهة بلغة .Cهذا هو الأسلوب الذي سيتبعه هذا الكتاب. أحيانًا، يُستخدم الاختصار PODلوصف بنية شبيهة بلغة ،Cأي بنية لا تحتوي على دوال أعضاء أو منشئات أو مدمرات. وهو يرمز إلى بيانات بسيطة.

سي++

Remember

في ،++Cيحدد إعلان الهيكل نوع الفئة.

الاتحادات والفئات مترابطة. كما هو الحال في البنية، يمكن استخدام الاتحاد لتعريف فئة. في لغة ،++Cقد تحتوي الاتحادات على دوال أعضاء ومتغيرات. كما قد تتضمن منشئات ومدمرات. يحتفظ الاتحاد في ++Cبجميع ميزاته المشابهة للغة ،Cوأهمها أن جميع عناصر البيانات تتشارك الموقع نفسه في الذاكرة. وكما هو الحال في البنية، تكون أعضاء الاتحاد عامة افتراضيًا ومتوافقة تمامًا مع لغة .Cفي المثال التالي، يُستخدم الاتحاد لتبادل البايتات التي تُكوّن عددًا صحيحًا قصيرًا بدون إشارة .

(يفترض هذا المثال أن الأعداد الصحيحة القصيرة يبلغ طولها 2بايت.)

```
#include <iostream>باستخدام مساحة الاسم
std؛

swap_byte { void swap(); void set_byte(i اتحاد قصيرة غير موقعة);
void show_word();

حرف uقصير غير موقّع؛ حرف [2]cغير
موقّع؛
};

void swap_byte::swap()
{
```

Machine Translated by Google


```
                رسم بياني غير موقّع؛

        c[0] = c[1]؛ c[1] = t؛
                t = c[0]؛

}

                void swap_byte::show_word() {

        cout << u;
                }

                        { (قصيرة غير موقعةvoid swap_byte::set_byte(i

        و =أنا؛
        }

int main() {

        مبادلة البايت ب؛

        ب.swap(); ب.show_word();
                ب.set_byte(49034);


                العودة ؛0
                }
```

مثل البنية، يُعرّف إعلان الاتحاد في C++نوعًا خاصًا من الفئات. هذا يعني أن مبدأ التغليف محفوظ.

هناك العديد من القيود التي يجب مراعاتها عند استخدام اتحادات .C++

أولاً، لا يمكن للاتحاد أن يرث أي فئات أخرى من أي نوع. علاوة على ذلك، لا يمكن للاتحاد أن يكون فئة أساسية. لا يمكن للاتحاد أن يحتوي على دوال أعضاء افتراضية. (ستُناقش الدوال الافتراضية في الفصل .(17لا يمكن لأي متغيرات ثابتة أن تكون أعضاءً في الاتحاد. لا يمكن استخدام عضو مرجعي. لا يمكن للاتحاد أن يحتوي على أي كائن يُحمّل عامل = كعضو. وأخيرًا، لا يمكن لأي كائن أن يكون عضوًا في الاتحاد إذا كان للكائن دالة منشئ أو مُدمر صريحة.

كما هو الحال مع ،structيتم تطبيق مصطلح PODأيضًا بشكل شائع على النقابات التي لا تحتوي على وظائف أعضاء أو منشئين أو مدمرين.

النقابات المجهولة

يوجد نوع خاص من الاتحادات في لغة C++يُسمى الاتحاد المجهول. لا يتضمن هذا الاتحاد اسم نوع، ولا يُمكن الإعلان عن أي كائنات تابعة له.

Machine Translated by Google

بدلاً من ذلك، يُخبر الاتحاد المجهول المُجمِّع بأن متغيراته الأعضاء ستتشارك الموقع نفسه. ومع ذلك، تتم الإشارة إلى المتغيرات نفسها مباشرةً، دون استخدام صيغة عامل النقطة الاعتيادية. على سبيل المثال، لننظر إلى هذا البرنامج:

```
#تضمين <iostream>
<cstring> #include باستخدام مساحة الاسم
std؛

int main() {

//قم بتعريف اتحاد مجهول ; { [4]; char s double d; long l; } union {

//الآن، قم بالإشارة إلى عناصر الاتحاد مباشرةً ؛100000 = l

cout << l؛
د ؛123.2342 =
cout << d؛
strcpy(s, "hi");
cout << s;

العودة ؛0
}
```

كما ترى، تتم الإشارة إلى عناصر الاتحاد كما لو كانت مُعلنة كمتغيرات محلية عادية. في الواقع، بالنسبة لبرنامجك، هذه هي الطريقة التي ستستخدمها بها بالضبط. علاوة على ذلك، على الرغم من تعريفها ضمن إعلان اتحاد ، إلا أنها تقع على نفس مستوى النطاق كأي متغير محلي آخر ضمن الكتلة نفسها. هذا يعني أن أسماء أعضاء الاتحاد المجهول يجب ألا تتعارض مع مُعرفات أخرى معروفة ضمن النطاق نفسه.

تنطبق جميع القيود المتعلقة بالنقابات على النقابات المجهولة، مع هذه الإضافات.
أولاً، يجب أن تكون العناصر الوحيدة المضمنة في اتحاد مجهول هي البيانات. لا يُسمح بأي دوال أعضاء. لا يمكن للاتحادات المجهولة أن تحتوي على عناصر خاصة أو محمية . وأخيرًا، يجب تحديد الاتحادات المجهولة العالمية على أنها ثابتة.

الدوال الصديقة: يُمكن منح دالة غير عضو حق الوصول إلى العناصر الخاصة للفئة باستخدام دالة صديقة. تتمتع الدالة الصديقة بحق الوصول إلى جميع العناصر الخاصة والمحمية .

Machine Translated by Google

للفئة التي تُعدّ دالة صديقة لها. لإعلان دالة صديقة ، أدرج نموذجها الأولي داخل الفئة، واسبقه بالكلمة المفتاحية "صديق".
لنفترض هذا البرنامج:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

عام:myclass { int a, b; الصف

sum(myclass x); void set_ab(int i, int j); }; صديق ;int

void myclass::set_ab(int i, int j) {

أ =ي؛ ب =ي؛

}

int sum(myclass x) { //ملاحظة: sum()ليست دالة عضو في أي فئة.

*/لأن sum()صديق لـ ،myclassفيمكنه
الوصول مباشرة إلى أ و ب. */

العودة ؛xa + xb
}

int main() {

صفي ؛n

ن.set_ab(3، 4)؛

مجموع >>مجموع (ن)؛

العودة ؛0
}
```

في هذا المثال، دالة ( )sumليست عضوًا في .myclassومع ذلك، لا تزال تتمتع بصلاحية الوصول الكامل إلى أعضائها الخاصين. لاحظ أيضًا أنه يتم استدعاء دالة ( )sumدون استخدام عامل النقطة. ولأنها ليست دالة عضو، فلا يلزم (بل قد لا يلزم) أن تكون مؤهلة باسم الكائن.

Machine Translated by Google

على الرغم من عدم وجود فائدة تُرجى من جعل ()sumدالة صديقة بدلًا من دالة عضو في myclass، إلا أن هناك بعض الحالات التي تكون فيها الدوال الصديقة قيّمة للغاية. أولًا، يمكن أن تكون الدوال الصديقة مفيدة عند زيادة تحميل أنواع معينة من المُعاملات (انظر الفصل 14)ثانيًا، تُسهّل الدوال الصديقة إنشاء بعض أنواع دوال الإدخال/الإخراج (انظر الفصل 18)ثالثًا، قد تكون الدوال الصديقة مرغوبة، لأنه في بعض الحالات، قد تحتوي فئتان أو أكثر على عناصر مترابطة نسبيًا مع أجزاء أخرى من برنامجك. لنتناول هذا الاستخدام الثالث الآن.

للبدء، تخيل فئتين مختلفتين، يعرض كل منهما رسالة منبثقة على الشاشة عند حدوث حالات خطأ. قد ترغب أجزاء أخرى من برنامجك في معرفة ما إذا كانت رسالة منبثقة تُعرض حاليًا قبل الكتابة على الشاشة، وذلك لتجنب الكتابة فوق أي رسالة عن طريق الخطأ. على الرغم من إمكانية إنشاء دوال أعضاء في كل فئة تُرجع قيمة تُشير إلى ما إذا كانت الرسالة نشطة، إلا أن هذا يعني تكلفة إضافية عند التحقق من الشرط (أي استدعاء دالتين، وليس واحدة فقط). إذا كان من الضروري التحقق من الشرط بشكل متكرر، فقد لا يكون هذا التكلفة الإضافية مقبولة. مع ذلك، باستخدام دالة مُشابهة لكل فئة، يُمكن التحقق من حالة كل كائن عن طريق استدعاء هذه الدالة فقط.

في مثل هذه الحالات، تتيح لك الدالة الصديقة إنشاء شيفرة برمجية أكثر كفاءة. يوضح البرنامج التالي هذا المفهوم:

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

int INUSE = 1؛ ثابت int IDLE = 0؛ ثابت

الفئة // ؛C2إعلان أمامي

الفئة // ؛C1 { int status إذا تم إيقاف تشغيله، INUSE إذا تم تشغيله على الشاشة // ... ؛(set_status(int state
public: void صديق ؛int idle(C1 a، C2 b)

};

الفئة // ؛C2 { int status إذا تم إيقاف تشغيله، INUSE إذا تم تشغيله على الشاشة // ... ؛(set_status(int state
public: void صديق ؛int idle(C1 a، C2 b)

};
```

Machine Translated by Google


```
void C1::set_status(int state) {
  الحالة =الدولة؛
}

void C2::set_status(int state) {
  الحالة =الدولة؛
}

int idle(C1 a, C2 b) {
  إذا (a.status || b.status)يتم إرجاع 0؛وإلا يتم إرجاع 1؛
}

int main() {
  ج1 س؛
  C2و؛

  sutats_tes.x(معرف); sutats_tes.y(معرف);

  إذا (idle(x, y)) cout << "يمكن استخدام الشاشة. \n";وإلا cout << "قيد الاستخدام:\n";

  x.set_status(INUSE);

  إذا (idle(x, y)) cout << "يمكن استخدام الشاشة. \n";وإلا cout << "قيد الاستخدام:\n";

  العودة 0؛
}
```

لاحظ أن هذا البرنامج يستخدم إعلانًا أماميًا (يُسمى أيضًا مرجعًا أماميًا) للفئة C2. هذا ضروري لأن إعلان idle() داخل C1 يشير إلى C2 قبل إعلانه. لإنشاء إعلان أمامي للفئة، استخدم النموذج الموضح في هذا البرنامج.

قد يكون صديق من فئة ما عضوًا في فئة أخرى. على سبيل المثال، إليك
تمت إعادة كتابة البرنامج السابق بحيث يكون idle( ) عضوًا في C1:

Machine Translated by Google


```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

int INUSE = 1 ؛ثابت int IDLE = 0 ؛ ثابت

الفئة // ؛C2إعلان أمامي

الفئة // IDLE int status; // ؛C1 { إذا تم إيقاف تشغيله، INUSEإذا تم عرضه على الشاشة // ... // ; (C2 b)int idle(int
state);
public: void set_status(int) الآن عضو في C1

};

الفئة // IDLE int status؛ // ؛C2 { إذا تم إيقاف تشغيله، INUSEإذا تم تشغيله على الشاشة // ... ؛set_status(int state)
public: void صديق ؛int C1::idle(C2 b)

};

void C1::set_status(int state) {

الحالة =الدولة؛
}

void C2::set_status(int state) {

الحالة =الدولة؛
}

C2 int C1::idle(C2 b) { ولكنه صديق لـ ،C1// idle() هو عضو في

إذا (الحالة b.status) || يتم إرجاع ؛0وإلا يتم إرجاع ؛1

}

int main() {
```

Machine Translated by Google


```
ج1 س؛
C2و؛

sutats_tes.x(معرف)؛ sutats_tes.y(معرف)؛

x.set_status(INUSE);"قيد الاستخدام.\n"; else cout << "يمكن استخدام الشاشة. if(x.idle(y)) cout <<
\n";

\n";"قيد الاستخدام.\n"; else cout << "يمكن استخدام الشاشة. if(x.idle(y)) cout <<

العودة ؛0
}
```

لأن idle()عضو في C1، يمكنه الوصول مباشرةً إلى متغير حالة كائنات النوع C1. وبالتالي، يلزم تمرير كائنات النوع C2فقط إلى idle( ).

هناك قيدان مهمان ينطبقان على الدوال الصديقة . أولاً، مشتق
لا ترث الفئة الدوال الصديقة . ثانيًا، لا يجوز أن تحتوي الدوال الصديقة على مُحدد فئة تخزين. أي أنه لا يجوز إعلانها كـ ثابتة أو خارجية.

الفئات الصديقة: من الممكن أن تكون فئة صديقة لفئة أخرى. في هذه الحالة، تتمتع الفئة الصديقة وجميع دوالها الأعضاء بالوصول إلى العناصر الخاصة المُعرّفة داخل الفئة الأخرى. على سبيل المثال،

```
//استخدام فئة صديقة.
#تضمين <iostream>
استخدام مساحة الاسم ؛std

TwoValues { الفئة

int a؛ int b؛عام:

Min؛ فئة صديقةTwoValues(int i, int j) { a = i; b = j; }

};

الصف مين {
```

Machine Translated by Google


```
عام:
nimint(قيمتان ;x)
};

nim::niMint(قيمتان { (x
xa < xb ؟ xa : xb; العودة
}

int main() {

قيمتان ؛(ob(10, 20
أنا أكون؛

cout << m.min(ob);

العودة ؛0
}
```

في هذا المثال، تتمتع الفئة Minبالقدرة على الوصول إلى المتغيرات الخاصة a وbالمعلنة داخل فئة . TwoValues

من المهم أن نفهم أنه عندما تكون إحدى الفئات صديقة لفئة أخرى، فهذا يعني أنها لا تمتلك سوى الوصول إلى الأسماء المُعرّفة ضمن الفئة الأخرى. لا يرث الفئة الأخرى.

على وجه التحديد، أعضاء الفئة الأولى لا يصبحون أعضاء في فئة الأصدقاء .

نادرًا ما تُستخدم فئات الأصدقاء. فهي مدعومة للتعامل مع حالات خاصة معينة.

الدوال المضمنة: هناك ميزة مهمة في لغة ،C++تُسمى الدالة المضمنة، وتُستخدم عادةً مع الفئات. بما أن بقية هذا الفصل (وبقية الكتاب) ستُستخدم بكثرة، فسيتم هنا دراسة الدوال المضمنة.

في ،C++يمكنك إنشاء وظائف قصيرة لا يتم استدعاؤها فعليًا؛ بل يتم استدعاء الكود الخاص بها يتم توسيع الدالة سطريًا عند كل استدعاء. تشبه هذه العملية استخدام ماكرو شبيه بالدالة. لتوسيع الدالة سطريًا بدلًا من استدعائها، اسبق تعريفها بالكلمة المفتاحية . inlineعلى سبيل المثال، في هذا البرنامج، يتم توسيع الدالة ( )maxسطريًا بدلًا من استدعائها:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std
```

Machine Translated by Google


```
int max(int a, int b) {

  العودة a>b؟ a : b;
}

int main() {

  cout << "الحد الأقصى (10، 20): " << max(10, 20) << endl;
  cout << "الحد الأقصى (88، 99): " << max(88, 99) << endl;

  العودة 0؛
}
```

فيما يتعلق بالمترجم، فإن البرنامج السابق يعادل هذا البرنامج:

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

int main() {

  cout << (10>20 ؟ 10 : 20);
  cout << (88<99 ؟ 99 : 88) << endl;

  العودة 0؛
}
```

السبب وراء كون **الوظائف** المضمنة إضافة مهمة إلى C++هو أنها تسمح
أنت لإنشاء كود فعال للغاية. نظرًا لأن الفئات تتطلب عادةً العديد من وظائف الواجهة التي يتم تنفيذها بشكل متكرر (والتي توفر الوصول إلى البيانات الخاصة)، فإن كفاءة هذه الوظائف أمر بالغ الأهمية. كما تعلم على الأرجح، في كل مرة يتم فيها استدعاء دالة، يتم توليد قدر كبير من النفقات العامة بواسطة آلية الاستدعاء والإرجاع. عادةً، يتم دفع الوسائط إلى المكدس ويتم حفظ السجلات المختلفة عند استدعاء دالة، ثم يتم استعادتها عند إرجاع الدالة. تكمن المشكلة في أن هذه التعليمات تستغرق وقتًا. ومع ذلك، عند توسيع دالة في السطر، لا تحدث أي من هذه العمليات. على الرغم من أن توسيع استدعاءات الوظائف في السطر يمكن أن ينتج أوقات تشغيل أسرع، إلا أنه يمكن أن يؤدي أيضًا إلى حجم كود أكبر بسبب الكود المكرر. لهذا السبب، من الأفضل تضمين الدوال الصغيرة جدًا فقط. علاوة على ذلك، من الجيد أيضًا تضمين الدوال التي سيكون لها تأثير كبير على أداء برنامجك فقط.

كما هو الحال مع مُحدِّد السجل ، فإن "inline"هو في الواقع مجرد طلب، وليس أمرًا، للمُجمِّع. يمكن للمُجمِّع تجاهله. كما أن بعض المُجمِّعات قد لا تُضمِّنه.

Machine Translated by Google

جميع أنواع الدوال. على سبيل المثال، من الشائع ألا يُضمّن المُجمّع دالةً تكرارية. ستحتاج إلى مراجعة وثائق المُجمّع بحثًا عن أي قيود على التضمين. تذكر، إذا تعذر تضمين دالة، فسيتم استدعاؤها ببساطة كدالة عادية.

قد تكون الدوال المضمنة دوالًا أعضاءً في الفئة. على سبيل المثال، هذا برنامج C++صالح تمامًا:


```
                    <iostream> تضمين#
                    std استخدام مساحة الاسم؛

a, b; public: void init(int i, int j); void show(); }; الصف
                    myclass { int

                                                    inline void myclass::init(int i, int j) { //إنشاء دالة مضمنة.

    أ =ي؛ ب =ي؛

}

                                                    inline void myclass::show() { //إنشاء دالة مضمنة أخرى.

                    cout << a <<                    << "\n"; ب>>
                                }

            int main() {

                    x؛ صفي

                x.init(10، 20)؛ x.show();

                    0؛ العودة
                                }
```

Note

الكلمة المفتاحية inlineليست جزءًا من مجموعة Cالفرعية من .C++وبالتالي، فهي غير مُعرّفة في .C89 ومع ذلك، فقد تمت إضافته بواسطة .C99

Machine Translated by Google

تعريف الدوال المضمنة داخل فئة: من الممكن تعريف دوال قصيرة بشكل كامل داخل إعلان فئة. عند تعريف دالة داخل إعلان فئة، تُحوّل تلقائيًا إلى دالة مضمنة (إن أمكن)، ليس من الضروري (ولكن ليس خطأً) إضافة الكلمة المفتاحية inline إلى إعلانها . على سبيل المثال، يُعاد كتابة البرنامج السابق هنا باستخدام تعريفي ( )init و ( )show الموجودين في إعلان :myclass

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

عام:myclass { int a, b; الصف

//تلقائي مضمن
void init(int i, int j) { a=i; b=j; }
void show() { cout << a << " " << b << "\n"; }
};

int main()
{
صفي ؛x

x.init(10، 20)؛ x.show();

العودة ؛0
}
```

لاحظ تنسيق كود الدالة داخل .myclass لأن الدوال المضمنة هي غالبًا ما يكون هذا النمط من الترميز قصيرًا، وهو نمط شائع في الصفوف. مع ذلك، يمكنك استخدام أي صيغة تُفضّلها. على سبيل المثال، هذه طريقة صحيحة تمامًا لإعادة كتابة إعلان : myclass

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

عام:myclass { int a, b; الصف

//تلقائي مضمن
void init(int i, int j)
{
أ =أنا؛
```

Machine Translated by Google


```
ب =ج؛
}

() { عرض باطل

cout << a << "  " >>ب ;"\n" >>
}
};
```

من الناحية الفنية، يُعدّ تضمين دالة ( )showمحدود الفائدة، لأن الوقت الذي تستغرقه عبارة الإدخال/الإخراج (بشكل عام) يتجاوز بكثير تكلفة استدعاء دالة. ومع ذلك، من الشائع جدًا رؤية جميع الدوال الأعضاء القصيرة مُعرّفة داخل فئتها في برامج .C++ (في الواقع، من النادر رؤية دوال أعضاء قصيرة مُعرّفة خارج إعلانات فئتها في أكواد ++Cالمكتوبة باحترافية.)

يمكن أيضًا تضمين وظائف المنشئ والمدمر، إما بشكل افتراضي، أو إذا
يتم تعريفها ضمن فئتها، أو بشكل صريح.

منشئات ذات معلمات: من الممكن تمرير وسيطات إلى منشئات. عادةً، تساعد هذه الوسيطات في تهيئة الكائن عند إنشائه.

لإنشاء منشئ ذي معلمات، ما عليك سوى إضافة معلمات إليه كما تفعل مع أي دالة أخرى. عند تعريف نص المنشئ، استخدم هذه المعلمات لتهيئة الكائن. على سبيل المثال، إليك فئة بسيطة تتضمن منشئًا ذي معلمات:

```
<iostream> #تضمين
std؛ استخدام مساحة الاسم

myclass(int i, int j) {a=i; b=j;} void show() {cout << a << << b;} }; الصف
myclass { int a, b; public:

"  "

int main() {

ob(3، 5)؛ صفي

ob.show();

0؛ العودة
}
```

Machine Translated by Google

لاحظ أنه في تعريف myclass( )، يتم استخدام المعلمات i و j لإعطاء القيم الأولية لـ a و b.

يوضح البرنامج الطريقة الأكثر شيوعًا لتحديد الوسائط عند إعلان كائن يستخدم مُنشئًا مُعَلَّمًا. على وجه التحديد، هذه العبارة

```
ob(3, 4); صفي
```

يؤدي إلى إنشاء كائن يسمى ob ويمرر الوسيطتين 3 و 4 إلى معلمتي i و j في myclass( ). يمكنك أيضًا تمرير الوسائط باستخدام هذا النوع من الإعلان إفادة:

```
صفي ob = صفي (3, 4);
```

مع ذلك، فإن الطريقة الأولى هي الأكثر شيوعًا، وهي النهج المُتبع في معظم الأمثلة في هذا الكتاب. في الواقع، هناك فرق فني بسيط بين نوعي الإعلانات يتعلق بمنشئي النسخ. (سيتم مناقشة منشئي النسخ في الفصل 14).

هذا مثال آخر يستخدم مُنشئًا مُعَلَّمًا. يُنشئ هذا المُنشئ فئةً تُخزِّن معلوماتٍ حول كتب المكتبة.

```
#تضمين <iostream>
<cstring> #include باستخدام مساحة الاسم std؛

int CHECKED_OUT = 0؛ ثابت int IN = 1؛ ثابت

status;} void set_status(int s) {status = s;} void show(); }; فئة الكتاب
int status; public: book(char *n, char *t, int s); int get_status() {return
{char author[40]; char title[40];

*n, char *t, int s) { rahc)كتاب::كتاب

ypcrts(المؤلف، ن)؛
```

Machine Translated by Google


```
                ypcrts(العنوان، :t)
                الحالة =س؛
                }

            كتاب فارغ::عرض() {

                                                cout
                                                "يكون :"                كوت >>
        إذا (الحالة ==في) >> cout"في. :"\nوإلا >> cout"خارج.:"\n

}

                                                                int main() {

b2("Melville", "Moby Dick", CHECKED_OUT); الكتابlb1("Twain", "Tom Sawyer", IN); الكتاب

ب1.عرض(); ب2.عرض();

            العودة 0؛
                }
```

تعتبر المنشئات المعلمة مفيدة للغاية لأنها تسمح لك بتجنب وجود
لإجراء استدعاء دالة إضافية لتهيئة متغير واحد أو أكثر في كائن. كل استدعاء دالة يمكنك تجنبه يزيد من كفاءة برنامجك. لاحظ أيضًا أن الدالتين القصيرتين ( )get_statusو ( )set_statusمُعرّفتان سطريًا، ضمن فئة الكتاب . هذه ممارسة شائعة عند كتابة برامج .C++

المنشئون ذوو المعلمة الواحدة: حالة خاصة

إذا كان للمنشئ مُعامل واحد فقط، فهناك طريقة ثالثة لتمرير قيمة ابتدائية إليه. على سبيل المثال، انظر إلى البرنامج القصير التالي.

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

X { int a؛ الصف
عام:
```

Machine Translated by Google


```
X(int j) { a = j; } int geta() { return a; }
};
int main() {
  X ob = 99; // يمرر 99 إلى j
  cout << ob.geta(); // ينتج 99
  return 0; العودة
}
```

هنا، يأخذ مُنشئ X مُعاملًا واحدًا. انتبه جيدًا لكيفية إعلان ob في main() . في هذا الشكل من التهيئة، يُمرر 99 تلقائيًا إلى المُعامل j في مُنشئ X() . أي أن المُترجم يتعامل مع بيان الإعلان كما لو كان مكتوبًا كالتالي:

```
X ob = X(99); اثنان X
```

بشكل عام، عند وجود مُنشئ يتطلب مُعاملًا واحدًا فقط، يمكنك استخدام ob(i) أو ob = i لتهيئة الكائن. ويرجع ذلك إلى أنه عند إنشاء مُنشئ يتطلب مُعاملًا واحدًا، فإنك تُنشئ ضمنيًا تحويلًا من نوع هذا المُعامل إلى نوع الفئة.

تذكر أن البديل الموضح هنا ينطبق فقط على المنشئين الذين لديهم معلمة واحدة فقط.

أعضاء الفئة الثابتة: يمكن جعل كلٍّ من عناصر الدالة والبيانات في الفئة ثابتة. يشرح هذا القسم نتائج كلٍّ منهما.

عناصر البيانات الثابتة: عند إضافة كلمة "static" قبل إعلان متغير عضو ، فإنك تُخبر المُجمّع بوجود نسخة واحدة فقط من هذا المتغير، وأن جميع كائنات الفئة ستشاركه. بخلاف عناصر البيانات العادية، لا تُنشأ نسخ فردية من متغير عضو ثابت لكل كائن. مهما كان عدد كائنات الفئة المُنشأة، توجد نسخة واحدة فقط من عنصر البيانات الثابت . وبالتالي، تستخدم جميع كائنات تلك الفئة نفس المتغير. تُهيأ جميع المتغيرات الثابتة إلى الصفر قبل إنشاء الكائن الأول.

Machine Translated by Google

عندما تعلن عن عنصر بيانات ثابت داخل فئة، فأنت لا تقوم بتعريفه. (هذا
(أي أنك لا تُخصّص مساحة تخزين له). بدلًا من ذلك، يجب عليك توفير تعريف عام له في مكان آخر، خارج الفئة. يتم ذلك بإعادة
إعلان المتغير الثابت باستخدام مُعامل حل النطاق لتحديد الفئة التي ينتمي إليها. يؤدي هذا إلى تخصيص مساحة تخزين للمتغير.
(تذكر أن إعلان الفئة هو ببساطة بناء منطقي لا وجود له في الواقع).

لفهم استخدام وتأثير عنصر البيانات الثابتة ، ضع في اعتبارك هذا البرنامج:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

الصف المشترك {
void set(int i، int j) {a=i؛ b=j؛} void show(); }؛ :عامint a؛ int b؛ ثابت

int shared::a; // a تحديدint

void shared::show() {
a: هذا ثابت"cout << " ؛<<
b: " cout << "\n"; هذا غير ثابت\n"cout << ؛>>ب

}

int main() {
x، y؛ مشترك

x.set(1, 1); // aتعيين إلى x.show(); 1

y.set(2, 2); // aتغيير إلى y.show(); 2

/* x.show(); هنا، تم تغيير a لكل من x و y لأن a مشتركة بين كلا الكائنين. */

0؛ العودة
}
```

Machine Translated by Google

يعرض هذا البرنامج الناتج التالي عند تشغيله.

```
هذا ثابت أ: 1
هذا غير ثابت ب: 1
هذا ثابت أ: 2
هذا غير ثابت ب: 2
هذا ثابت أ: 2
هذا غير ثابت ب: 1
```

لاحظ أن العدد الصحيح aمُعلن داخل وخارج المُشترك . كما ذكرنا سابقًا، هذا ضروري لأن إعلان المُشترك لا يُخصص مساحة تخزين.

Note

لتسهيل الأمر، لم تكن الإصدارات القديمة من لغة C++تتطلب الإعلان الثاني عن متغير عضو ثابت . إلا أن هذه السهولة أدت إلى تناقضات خطيرة، وتم التخلي عنها منذ عدة سنوات. مع ذلك، قد تجد أكوادًا قديمة من لغة C++لا تعيد الإعلان عن متغيرات العضو الثابتة . في هذه الحالات، ستحتاج إلى إضافة التعريفات المطلوبة.

يوجد متغير عضو ثابت قبل إنشاء أي كائن من فئته. على سبيل المثال،

في البرنامج القصير التالي، aعام وثابت . وبالتالي ، يمكن الوصول إليه مباشرةً في . ( )mainعلاوة على ذلك، بما أن aموجود قبل إنشاء كائن ، sharedيمكن إعطاء aقيمة في أي وقت. كما يوضح هذا البرنامج، فإن قيمة a لا تتغير بإنشاء الكائن .xلهذا السبب، يعرض كلا البيانين الناتجين القيمة نفسها: .99

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

الصف المشترك {
  عام: ثابت ؛int a

};

int shared::a; // تحديد a

int main()
{
  //قم بتهيئة a قبل إنشاء أي كائنات
  مشترك::أ ؛99 =

  " >> cout"هذه هي القيمة الأولية لـ: >>مشترك::أ؛
```

Machine Translated by Google


```
                cout << "\n";

        x: مشترك

                xa: هذا هو"cout <<  "  <<xa;

        0: العودة
        }
```

لاحظ كيف تتم الإشارة إلى aمن خلال استخدام اسم الفصل وحل النطاق
عامل. بشكل عام، للإشارة إلى عنصر ثابت بشكل مستقل عن الكائن، يجب عليك تحديده باستخدام اسم الفئة التي يكون عضوًا فيها.

أحد استخدامات متغير العضو الثابت هو توفير التحكم في الوصول إلى مورد مشترك تستخدمه جميع كائنات الفئة. على سبيل المثال، يمكنك إنشاء عدة كائنات، يحتاج كل منها إلى الكتابة إلى ملف قرص محدد. مع ذلك، من الواضح أنه لا يُسمح إلا لكائن واحد بالكتابة إلى الملف في كل مرة. في هذه الحالة، ستحتاج إلى إعلان متغير ثابت يُشير إلى وقت استخدام الملف ووقت فراغه. بعد ذلك، يستفسر كل كائن عن هذا المتغير قبل الكتابة إلى الملف. يوضح البرنامج التالي كيفية استخدام متغير ثابت من هذا النوع للتحكم في الوصول إلى مورد نادر:

```
        #تضمين <iostream>
        استخدام مساحة الاسم std:

الفئة cl{ ثابت intمورد؛ عام: void free_resource() ;int get_resource() { مورد ;} {0:
                                                                =

                                                int cl::resource; // تعريف المورد intالمورد

                                                int cl::get_resource() {

                                                إذا (المورد) قم بإرجاع // 0:المورد قيد الاستخدام بالفعل وإلا {

                                                المورد 1: =إرجاع // 1:المورد المخصص لهذا الكائن

    }
    }
```

Machine Translated by Google


```
int main() {

    cl ob1، ob2؛

                                                    ;"n\(ob1.get_resource()) cout << "ob1 إذا

                                                    ;"n\cout << "ob2 (()ecruoser_teg.2bo) إذا لم

                                                    // ob1.free_resource(); دع شخصًا آخر يستخدمه

    >> cout ((ob2.get_resource()if(ob2 يمكن لـ "ob2 الآن استخدام المورد\n";


    العودة 0؛
    }
```

استخدام آخر مثير للاهتمام لمتغير عضو ثابت هو تتبع الرقم
من كائنات من نوع فئة معين موجودة. على سبيل المثال،

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

Counter { public: static int count; الفئة

() { count++; } عداد
~Counter() { count--; } }; int Counter::count;



f(); باطل

int main(void) {

<< Counter::count << "\n"؛ cout << "o1 الكائنات الموجودة: "؛ cout << العداد
                                                    ";

cout << Counter::count << "\n"؛ cout << "o2 الكائنات الموجودة: "؛ cout << عداد
                                                    ";
```

Machine Translated by Google


```
"; cout << Counter::count << "\n"; :الكائنات الموجودة"f(); cout <<


    العودة ؛0
}

f() { باطل }

    درجة حرارة العداد؛
cout <<"الكائنات الموجودة: // "\n"; << Counter::count << "\n"; ;"يتم تدمير
                                    f() درجة الحرارة عند إرجاع

}
```

ينتج هذا البرنامج الناتج التالي.

```
الأشياء الموجودة: 1
الأشياء الموجودة: 2
الأشياء الموجودة: 3
الأشياء الموجودة: 2
```

كما ترى، يزداد عدد متغير العضو الثابت عند إنشاء كائن، وينخفض عند تدميره. بهذه الطريقة، يتتبع عدد الكائنات من نوع "Count "الموجودة حاليًا.

باستخدام متغيرات الأعضاء الثابتة ، يُمكنك الاستغناء تقريبًا عن أي حاجة للمتغيرات العالمية. تكمن مشكلة المتغيرات العالمية مقارنةً بالبرمجة الكائنية التوجه في أنها غالبًا ما تنتهك مبدأ التغليف.

الدوال العضوية الثابتة

يمكن أيضًا إعلان الدوال الأعضاء كدوال ثابتة. هناك عدة قيود مفروضة على الدوال الأعضاء الثابتة . لا يجوز لها الإشارة مباشرةً إلا إلى عناصر ثابتة أخرى في الفئة. (بالطبع، يمكن الوصول إلى الدوال والبيانات العامة بواسطة الدوال الأعضاء الثابتة).

لا تحتوي الدالة العضو الثابتة على مؤشر . this(انظر الفصل 13لمزيد من المعلومات حول هذا الموضوع). لا يمكن وجود إصدار ثابت وآخر غير ثابت للدالة نفسها. لا يمكن أن تكون الدالة العضو الثابتة افتراضية. وأخيرًا، لا يمكن إعلانها كدالة ثابتة أو متغيرة.

فيما يلي نسخة مُعدّلة قليلاً من برنامج الموارد المشتركة من القسم السابق. لاحظ أن ()get_resourceمُعلن الآن كدالة ثابتة. كما يوضح البرنامج، يُمكن استدعاء ()get_resourceإما بمفردها، أو بشكل مستقل عن أي كائن، أو باستخدام اسم الفئة ومُعامل حل النطاق، أو بالارتباط مع كائن.

Machine Translated by Google


```
#تضمين <iostream>
استخدام مساحة الاسم std؛

الفئة cl{ مورد intثابت؛

عام:
int get_resource(); void free_resource() { resource = 0; } }; ثابت

int cl::resource; // تعريف المورد

int cl::get_resource()
{
إذا (المورد) قم بإرجاع // 0؛ المورد قيد الاستخدام بالفعل
آخر {
المورد = 1؛
العودة // 1؛ الموارد المخصصة لهذا الكائن
}
}

int main()
{
cl ob1، ob2؛

get_resource() */ثابت لذا يمكن استدعاؤه مستقلاً
من أي كائن. */
إذا "ob1 << cout << (cl::get_resource())لديه مورد\n"؛

إذا لم(cl::get_resource()) "ob2 << cout رفض المورد\n";

ob1.free_resource();

// (ob2.get_resource())if لا يزال بإمكانك الاتصال باستخدام صيغة الكائن "ob2 << cout يمكنه الآن استخدام المورد\n";

العودة 0؛
}
```

في الواقع، دوال الأعضاء الثابتة لها تطبيقات محدودة، ولكن أحد استخداماتها الجيدة هو "التهيئة المسبقة" للبيانات الثابتة الخاصة قبل إنشاء أي كائن. على سبيل المثال، هذا برنامج C++ صالح تمامًا:

Machine Translated by Google


```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

static void init(int x) {i = x;} void show() {cout << i;} }; الفئة
                    static_type { static int i; public:

                                                    i تعريفint static_type::i; //

                                                                        int main() {

                                                static_type::init(100); //قم بتهيئة البيانات الثابتة قبل إنشاء الكائن

                                                        static_type x; x.show(); // 100يعرض

                                                                                0؛ العودة
                                                                                }
```

كقاعدة عامة، يُستدعى مُنشئ الكائن عند ظهوره، ويُستدعى مُدمره عند تدميره. سيتم هنا مناقشة توقيت حدوث هذين الحدثين تحديدًا.

يُنفَّذ مُنشئ الكائن المحلي عند مواجهة بيان إعلان الكائن. تُنفَّذ مُدمِّرات الكائنات المحلية بعكس ترتيب دوال المُنشئ.

تُنفَّذ مُنشئات الكائنات العالمية قبل بدء تنفيذ دالة . ()mainتُنفَّذ المُنشئات العالمية حسب ترتيب إعلانها، داخل الملف نفسه. لا يُمكن معرفة ترتيب تنفيذ المُنشئات العالمية الموزعة على عدة ملفات. تُنفَّذ مُدمِّرات الكائنات العالمية بترتيب عكسي بعد انتهاء دالة . ()main

يوضح هذا البرنامج متى يتم تنفيذ المنشئين والمدمرين:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std
```

Machine Translated by Google


```
الصف myclass{ عام:

int من؛ ~myclass(); id)؛
myclass(int

} glob_ob1(1)، glob_ob2(2)؛

myclass::myclass(int id) {

" who = id; جاري تهيئة"cout << << "\n"; >>معرف

}

صفي::~صفي() {

>>ظ"تدمير "

}

int main() {

local_ob1(3); صفي

>> cout"لن يتم عرض هذا السطر الأول.;"\n

local_ob2(4); صفي

العودة ؛0
}
```

ويعرض هذا الإخراج:

```
جاري التهيئة 1جاري التهيئة 2جاري التهيئة 3لن يتم عرض هذا في السطر الأول.

تهيئة 4تدمير 4تدمير 3تدمير 2تدمير 1
```

شيء واحد: بسبب الاختلافات بين المجمّعات وبيئات التنفيذ، قد ترى أو لا ترى آخر سطرين من الإخراج.

Machine Translated by Google

عامل حل النطاق: كما تعلم، يربط **عامل ::** اسم الفئة باسم العضو لإخبار المُجمِّع بالفئة التي ينتمي إليها العضو. ومع ذلك، لعامل حل النطاق استخدام آخر ذو صلة: إذ يسمح بالوصول إلى اسم في نطاق مُغلق "مُخفي" بإعلان محلي يحمل الاسم نفسه. على سبيل المثال، انظر إلى هذا المقطع:

```
int i; // iعالمي

f() { باطل

int i; // iمحلي

i = 10؛ // iيستخدم iالمحلي
.
.
.
}
```

كما يشير التعليق، فإن التعيين 10 = iيشير إلى الدالة iالمحلية . ولكن ماذا لو احتاجت الدالة ( )fللوصول إلى النسخة العامة من ؟iيمكن تحقيق ذلك بإضافة عامل :: قبل ، اكما هو موضح هنا.

```
int i; // iعالمي

f() { باطل

int i; // iمحلي

i = 10; // i::يشير الآن إلى iالعالمي
.
.
.
}
```

الفئات المتداخلة: من الممكن تعريف فئة داخل أخرى، مما يُنشئ **فئة متداخلة** . وبما أن **إعلان الفئة** يُعرّف نطاقًا، فإن الفئة المتداخلة لا تكون صالحة إلا ضمن نطاق الفئة المُحيطة بها. بصراحة، نادرًا ما تُستخدم الفئات المتداخلة. بفضل آلية الوراثة المرنة والفعّالة في لغة ،++Cتكاد تكون الحاجة إلى الفئات المتداخلة معدومة.

Machine Translated by Google

الفئات المحلية: يمكن تعريف فئة داخل دالة. على سبيل المثال، هذا برنامج C++ صالح:

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

باطل f();

int main() {

return 0; // myclass غير معروف هنا f();

}

باطل f() {

الصف myclass { int i؛ } عام:

void put_i(int n) { i=n; } int get_i() { return i; } } ob;

ob.put_i(10); cout << ob.get_i();

}
```

عندما يتم إعلان فئة داخل دالة، فهي معروفة فقط لتلك الدالة وغير معروفة خارجها.

تنطبق عدة قيود على الفئات المحلية. أولاً، يجب أن تكون جميع الدوال الأعضاء مُعرَّفة ضمن إعلان الفئة. لا يُمكن للفئة المحلية استخدام أو الوصول إلى المتغيرات المحلية للدالة المُعلَّنة فيها (باستثناء أن الفئة المحلية تتمتع بالوصول إلى المتغيرات المحلية الثابتة المُعلَّنة داخل الدالة أو تلك المُعلَّنة كـ extern). مع ذلك، يُمكنها الوصول إلى أسماء الأنواع والعدادات المُعرَّفة بواسطة الدالة المُحيطة.

لا يُسمح بإعلان متغيرات ثابتة داخل فئة محلية. وبسبب هذه القيود، لا تُستخدم الفئات المحلية في برمجة C++.

تمرير الكائنات إلى الوظائف

يمكن تمرير الكائنات إلى الدوال بنفس طريقة أي نوع آخر من المتغيرات. تُمرر الكائنات إلى الدوال باستخدام آلية الاستدعاء بالقيمة القياسية. على الرغم من أن تمرير الكائنات سهل، إلا أن بعضها...

Machine Translated by Google

تحدث أحداث غير متوقعة تتعلق بالمنشئين والمدمرين. لفهم السبب، انظر إلى هذا البرنامج القصير.

```
//تمرير كائن إلى دالة.
#تضمين <iostream>
استخدام مساحة الاسم ؛std

set_i(int n) { i=n؛ } int get_i() :عامmyclass { int i؛ الصف
i؛ } إرجاع { i؛ }myclass(int n)؛ ~myclass()؛ void

};

myclass::myclass(int n) {

"جاري البناء"i = n; cout <<

<<أنا "\n"; <<
}

صفي::~صفي() {

<<أنا تدمير "
}

ob)؛ صفي)fvoid

int main() {

o(1)؛ صفي

main: "; cout << o.get_i() << "\n"; في هذا هو "f(o); cout <<

0؛ العودة
}

void f(myclass ob) {

ob.set_i(2);
```

Machine Translated by Google


```
cout << "هذا هو الموقع المحلي: " << o.get_i() << "\n";
}
```

يُنتج هذا البرنامج الناتج التالي:

```
بناء 1
هذا هو الموقع المحلي 2 :i
تدمير 2
هذا هو أنا في الرئيسي: 1
تدمير 1
```

كما هو موضح في المخرجات، هناك استدعاء واحد للمنشئ، والذي يحدث عند إنشاء oفي ، ( )mainولكن هناك استدعائان للمدمر. لنرى سبب ذلك.
عندما يتم تمرير كائن إلى دالة، يتم عمل نسخة من هذا الكائن (وهذه النسخة
يصبح المُعامل في الدالة). هذا يعني ظهور كائن جديد. عند انتهاء الدالة، تُدمَّر نسخة المُعامل (أي المُعامل). هذا يُثير سؤالين أساسيين: أولًا، هل يُستدعى مُنشئ الكائن عند إنشاء النسخة؟ ثانيًا، هل يُستدعى مُدمر الكائن عند تدمير النسخة؟ قد تُفاجئك الإجابات للوهلة الأولى.

عندما يتم عمل نسخة من وسيطة أثناء استدعاء وظيفة، فإن المنشئ العادي
لا يتم استدعاؤه. بدلاً من ذلك، يتم استدعاء مُنشئ نسخ الكائن . يُحدد مُنشئ النسخ كيفية إنشاء نسخة من الكائن. كما هو موضح في الفصل ،14يمكنك تعريف مُنشئ نسخ صراحةً للفئة التي تنشئها. ومع ذلك، إذا لم تُعرّف الفئة مُنشئ نسخ صراحةً، كما هو الحال هنا، فإن C++توفر واحدًا بشكل افتراضي. يُنشئ مُنشئ النسخ الافتراضي نسخة بتية (أي متطابقة) من الكائن. من السهل فهم سبب إنشاء نسخة بتية إذا فكرت في الأمر. نظرًا لأن المُنشئ العادي يُستخدم لتهيئة بعض جوانب الكائن، فلا يجب استدعاؤه لإنشاء نسخة من كائن موجود بالفعل. ستؤدي مثل هذه الدعوة إلى تغيير محتويات الكائن.

عند تمرير كائن إلى دالة، فأنت تريد استخدام الحالة الحالية للكائن، وليس حالته الأولية.

مع ذلك، عند انتهاء الدالة وتدمير نسخة الكائن المستخدمة كوسيطة، يُستدعى المُدمِّر . هذا ضروري لأن الكائن قد خرج عن نطاقه. لهذا السبب، استدعى البرنامج السابق مُدمِّرين.

الأول كان عندما خرج مُعامل ( )fعن نطاقه. والثاني عندما تم تدمير oداخل ( )mainعند انتهاء البرنامج.

للتلخيص: عند إنشاء نسخة من كائن لاستخدامها كوسيطة لدالة، لا يُستدعى المُنشئ العادي. بدلاً من ذلك، يُنشئ مُنشئ النسخ الافتراضي نسخة متطابقة بتًا بتًا. ومع ذلك، عند إتلاف النسخة (عادةً بخروجها عن نطاق الدالة عند عودتها)، يُستدعى المُدمر.

لأن مُنشئ النسخة الافتراضية يُنشئ نسخة طبق الأصل من الأصل، فقد يُسبب مشاكل أحيانًا. على الرغم من أن الكائنات تُمرَّر إلى الدوال باستخدام آلية تمرير معلمات الاستدعاء بالقيمة الاعتيادية، والتي نظريًا،

Machine Translated by Google

على الرغم من أن الدالة تحمي وتعزل وسيطة الاستدعاء، إلا أنه من الممكن حدوث تأثير جانبي قد يؤثر على الكائن المستخدم كوسيطة، أو حتى يتلفه. على سبيل المثال، إذا خصص كائن مستخدم كوسيطة ذاكرة وحررها عند تدميره، فإن نسخته المحلية داخل الدالة ستحرر نفس الذاكرة عند استدعاء مُدمِّرها. سيؤدي هذا إلى تلف الكائن الأصلي وجعله عديم الفائدة عمليًا.

ولمنع حدوث هذا النوع من المشكلات، ستحتاج إلى تعريف عملية النسخ عن طريق إنشاء منشئ نسخ للفئة، كما هو موضح في الفصل 14.

إرجاع الكائنات

قد تُرجع الدالة كائنًا إلى المُستدعي. على سبيل المثال، هذا برنامج C++صالح:

```
//إرجاع الكائنات من الدالة.
#تضمين <iostream>
استخدام مساحة الاسم std؛

n) { i=n؛ } int get_i() { return i؛ } :عامmyclass { int i؛ الصف
                                    void set_i(int

};

                                    myclass إرجاع كائن من نوعmyclass f(); //

int main() {

    o؛ صفي

  o = f();

                cout << o.get_i() << "\n";

    0؛ العودة
    }

f() { صفي

  x؛ صفي

  x.set_i(1);
  x؛ العودة
    }
```

Machine Translated by Google

عند إرجاع كائن بواسطة دالة، يُنشأ تلقائيًا كائن مؤقت يحمل قيمة الإرجاع. هذا الكائن هو ما تُرجعه الدالة فعليًا. بعد إرجاع القيمة، يُدمر هذا الكائن. قد يُسبب تدمير هذا الكائن المؤقت آثارًا جانبية غير متوقعة في بعض الحالات. على سبيل المثال، إذا كان الكائن المُعاد بواسطة الدالة يحتوي على مُدمر يُحرر ذاكرة مُخصصة ديناميكيًا، فسيتم تحرير هذه الذاكرة حتى لو كان الكائن الذي يستقبل قيمة الإرجاع لا يزال يستخدمها. هناك طرق للتغلب على هذه المشكلة تتضمن زيادة تحميل مُعامل التعيين (انظر الفصل 15)وتحديد مُنشئ نسخ (انظر الفصل 14).

تعيين الكائن

بافتراض أن كلا الكائنين من نفس النوع، يمكنك تعيين كائن إلى آخر.
يؤدي هذا إلى نسخ بيانات الكائن الأيمن إلى بيانات الكائن الأيسر. على سبيل المثال، يعرض هذا البرنامج :99

```
//تعيين الكائنات.
#تضمين <iostream>
استخدام مساحة الاسم ؛std

{ i=n؛ } int get_i() { return i؛ } :عامmyclass { int i؛ الصف
                                void set_i(int n)




};

int main() {

        ob1، ob2؛ صفي

                ob2إلى ob1 تعيين البيانات من ob1.set_i(99); ob2 = ob1; //

                                                ob2's i: " << ob2.get_i();

العودة ؛0
}
```

افتراضيًا، تُخصَّص جميع البيانات من أحد الكائنين إلى الآخر باستخدام نسخة بت بت. ومع ذلك، من الممكن زيادة تحميل مُعامل التعيين وتحديد إجراء تعيين آخر (انظر الفصل 15).

Machine Translated by Google

The Complete Reference



الفصل 13

المصفوفات والمؤشرات والمراجع والتخصيص الديناميكي المشغلين

جميع الحقوق محفوظة ٢٠٠٣ ©لشركة ماكجرو هيل. انقر هنا للاطلاع على شروط الاستخدام.

Machine Translated by Google

في الجزء الأول، تم فحص المؤشرات والمصفوفات فيما يتعلق بـ C++ المضمنة
أنواع البيانات المجردة (ADTs) والوظائف. يختتم الفصل
فحص مشغلي التخصيص الديناميكي في C++.

مصفوفات الكائنات

في لغة C++، من الممكن استخدام مصفوفات من الكائنات. قواعد تعريف واستخدام مصفوفات الكائنات مطابقة تمامًا لقواعد أي نوع آخر من المصفوفات. على سبيل المثال، يستخدم هذا البرنامج مصفوفة من الكائنات ثلاثية العناصر:

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

الصف cl { int i؛ عام: cl { i=j؛ } int get_i() { return i؛ } void set_i(int j) { i=j؛ }
};

int main() {
cl ob[3]; int i;

لـ(i=0؛ i<3؛ i++) ob[i].set_i(i+1)؛

(i = 0؛ i <3؛ i++) cout << ob[i].get_i() << "\n"؛ بالنسبة إلى

العودة 0؛
}
```

يعرض هذا البرنامج الأرقام 1 و 2 و 3 على الشاشة.
إذا قامت فئة بتعريف منشئ معلمات، فيمكنك تهيئة كل كائن في مصفوفة من خلال تحديد قائمة تهيئة، تمامًا كما تفعل مع أنواع أخرى من المصفوفات.
مع ذلك، سيُحدَّد الشكل الدقيق لقائمة التهيئة بناءً على عدد المعلمات المطلوبة من قِبل مُنشئات الكائن. بالنسبة للكائنات التي تحتوي مُنشئاتها على مُعامل واحد فقط، يُمكنك ببساطة تحديد قائمة بالقيم الأولية، باستخدام صيغة تهيئة المصفوفة الاعتيادية. عند إنشاء كل عنصر في المصفوفة، تُضاف قيمة من

Machine Translated by Google

يتم تمرير القائمة إلى مُعامل المُنشئ. على سبيل المثال، إليك نسخة مختلفة قليلاً من البرنامج السابق تستخدم تهيئة:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

int get_i() { return i؛ } المنشئcl(int j) { i=j؛ } // :عامcl { int i؛ الصف

};

int main() {

int i؛ المُبدئياتcl ob[3] = {1, 2, 3}; //

(i = 0؛ i <3؛ i++) cout << ob[i].get_i() << "\n"؛ بالنسبة إلى

العودة ؛0
}
```

كما في السابق، يعرض هذا البرنامج الأرقام 1و 2و 3على الشاشة.
في الواقع، فإن بناء الجملة التهيئة الموضح في البرنامج السابق هو اختصار لهذا النموذج الأطول:

```
cl ob[3] = { cl(1)، cl(2)، cl(3) };
```

هنا، يتم استدعاء مُنشئ الدالة clصراحةً. وبالطبع، يُعدّ الشكل المختصر المستخدم في البرنامج أكثر شيوعًا. يعمل الشكل المختصر بفضل التحويل التلقائي المُطبّق على المُنشئات التي تأخذ مُعاملًا واحدًا فقط (انظر الفصل .١٢وبالتالي، لا يُمكن استخدام الشكل المختصر إلا لتهيئة مصفوفات الكائنات التي تتطلب مُنشئاتها مُعاملًا واحدًا فقط.

إذا كان منشئ الكائن يتطلب وسيطتين أو أكثر، فسوف يتعين عليك استخدام
نموذج تهيئة أطول. على سبيل المثال،

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std
```

Machine Translated by Google


```
int get_i() {return i;} int get_h() {return h;} منشئ مع معاملين cl { int h; int i; public: cl(int j, int k) { h=j; i=k; } // الفئة

};

int main() {

cl ob[3] = {
cl(3, 4), cl(5, 6) تهيئة cl(1, 2), //

};

int i;

لـ(i=0; i<3; i++) {
cout << ob[i].get_h();
كوت << ", ";
cout << ob[i].get_i() << "\n";
}

العودة 0;
}
```

هنا، يحتوي مُنشئ الدالة cl على مُعاملَين، وبالتالي يتطلب وسيطَين. هذا يعني أنه لا يُمكن استخدام صيغة التهيئة المختصرة، ويجب استخدام الصيغة الطويلة الموضحة في المثال.

إنشاء المصفوفات المُهيأة وغير المُهيأة

تحدث حالة خاصة إذا كنت تنوي إنشاء مصفوفات من الكائنات مُهيأة وغير مُهيأة. خذ الفئة التالية في الاعتبار.

```
cl(int j) { i=j; } عام: cl { int i; الصف
```

Machine Translated by Google


```
int get_i() { return i; }
};
```

هنا، يتطلب المُنشئ المُعرّف بواسطة cl مُعاملًا واحدًا. هذا يعني أن أي مصفوفة مُعلنة من هذا النوع يجب تهيئةُها. أي أنه يمنع إعلان المصفوفة هذا:

```
cl a[9]; // خطأ، يتطلب المنشئ مُبدئات
```

سبب عدم صحة هذه العبارة (كما هو مُعرّف حاليًا) هو أنها تُشير إلى أن cl لديها مُنشئ بدون مُعاملات لعدم تحديد مُبدئات. مع ذلك، في وضعها الحالي، cl لا تحتوي على مُنشئ بدون مُعاملات. نظرًا لعدم وجود مُنشئ صحيح يتوافق مع هذا الإعلان، سيُبلغ المُترجم عن خطأ.

لحل هذه المشكلة، يجب زيادة تحميل المُنشئ، بإضافة مُنشئ لا يأخذ أي مُعاملات، كما هو موضح لاحقًا. بهذه الطريقة، يُسمح لكلٍّ من المصفوفات المُهيأة وغير المُهيأة.

```
class cl { int i; public: cl() { i=0; } // تم استدعاؤها للمصفوفات المهيئة cl(int j) { i=j; } // تم استدعاؤها للمصفوفات غير المهيئة { return i; } get_i() int

};
```

نظرًا لهذه الفئة، فإن كلا البيانين التاليين مسموح بهما:

```
cl a1[3] = {3, 5, 6}; // تم تهيئة
cl a2[34]; // غير مهيأ
```

مؤشرات إلى الكائنات

كما يمكنك استخدام مؤشرات لأنواع أخرى من المتغيرات، يمكنك استخدام مؤشرات للكائنات. عند الوصول إلى عناصر فئة معينة باستخدام مؤشر لكائن، استخدم عامل السهم (->)بدلاً من عامل النقطة. يوضح البرنامج التالي كيفية الوصول إلى كائن باستخدام مؤشر له:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std
```

Machine Translated by Google


```
i؛ } إرجاع { }cl(int j) { i=j؛ } int get_i() :عامcl { int i؛ الصف

};

int main() {

ص*cl ob(88)،؛

ob الحصول على عنوان p = &ob; //

استخدام cout << p->get_i(); // لاستدعاء get_i() <-

العودة 0؛
}
```

كما تعلم، عندما يتم زيادة المؤشر، فإنه يشير إلى العنصر التالي من نوعه.
على سبيل المثال، سيشير مؤشر عدد صحيح إلى العدد الصحيح التالي. عمومًا، جميع عمليات المؤشر الحسابية مرتبطة بنوعه الأساسي (أي مرتبطة بنوع البيانات التي يُشير إليها المؤشر). وينطبق الأمر نفسه على مؤشرات الكائنات.

على سبيل المثال، يستخدم هذا البرنامج مؤشرًا للوصول إلى جميع العناصر الثلاثة للمصفوفة obبعد تعيين عنوان بداية aلـob:

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

{ i = 0؛ } cl(int j) { i = j؛ } int get_i() :عامcl { int i؛ الصف
i؛ } إرجاع { }cl()

};

int main() {

cl ob[3] = {1، 2، 3}؛
```

Machine Translated by Google


```
                                        cl *p؛ int i؛


for(i=0; i<3; i++) { الحصول على بداية المصفوفة p = ob; //

    // ؛cout << p->get_i() << "\n"; p++ أشر إلى الكائن التالي

}

العودة ؛0
}
```


يمكنك تعيين عنوان عضو عام لكائن إلى مؤشر ثم
الوصول إلى هذا العنصر باستخدام المؤشر. على سبيل المثال، هذا برنامج C++صالح يعرض الرقم 1على الشاشة:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

int i؛ cl(int j) { i = j؛ } }؛ عام: cl{ الصف


int main() {

cl ob(1); int *p;


                    ob.i الحصول على عنوان p = &ob.i; //

                    p عبر ob.i الوصول إلى cout << *p; //

العودة ؛0
}
```

لأن pيشير إلى عدد صحيح، يُعلن عنه كمؤشر عدد صحيح. لا أهمية لكون aعضوًا في obفي هذه الحالة.

Machine Translated by Google

فحص نوع المؤشرات في لغة ++C

هناك أمر مهم يجب فهمه حول المؤشرات في لغة ++C: لا يُمكنك تعيين مؤشر إلى آخر إلا إذا كان نوعا المؤشر متوافقين. على سبيل المثال، لنفترض أن:

```
int *pi; float *pf;
```

في C++، التعيين التالي غير قانوني:

```
pi = pf; // خطأ -- عدم تطابق النوع
```

بالطبع، يمكنك تجاوز أي عدم توافق في النوع باستخدام التحويل، ولكن القيام بذلك يتجاوز آلية التحقق من النوع في C++.

عندما يتم استدعاء دالة عضو، يتم تمرير وسيطة ضمنية إليها تلقائيًا وهي عبارة عن مؤشر إلى الكائن المستدعي (أي الكائن الذي يتم استدعاء الدالة عليه).

يُسمى هذا المؤشر بهذا الاسم. لفهم ذلك، لنفترض أولًا برنامجًا يُنشئ فئةً تُسمى pwr، تحسب نتيجة عددٍ مرفوعٍ إلى قوةٍ مُعينة:

```
#تضمين <iostream>
استخدام مساحة الاسم std;

الفئة pwr { double b; int e; double val; عام: pwr(قاعدة مزدوجة، int exp); double get_pwr () { إرجاع val; } };

rwp::rwp(قاعدة مزدوجة، int exp) {
ب = قاعدة؛
هـ = إكسب؛
val = 1؛ إذا (exp == 0) العودة؛ ل (؛ exp> 0؛ exp--) val = val * b؛
```

Machine Translated by Google


```
}

int main() {

x(4.0، 2)، y(2.5، 1)، z(5.7، 0)؛ الطاقة

<< cout << y.get_pwr() << cout << z.get_pwr() << "\n";
cout << x.get_pwr()

0؛ العودة
}
```

داخل دالة عضو، يمكن الوصول إلى عناصر الفئة مباشرةً، دون أي مؤهلات للكائن أو الفئة. وبالتالي، داخل الدالة ( )pwr، تكون العبارة

```
ب =قاعدة؛
```

يعني ذلك أن نسخة b المرتبطة بالكائن المستدعي ستُخصص لها القيمة الموجودة في base. مع ذلك، يمكن كتابة العبارة نفسها كما يلي:

```
هذا->ب =قاعدة؛
```

يشير مؤشر this إلى الكائن الذي استدعى الدالة ( )pwr. وبالتالي، يشير this –>b إلى نسخة الكائن من الدالة b. على سبيل المثال، إذا استُدعيت الدالة ( )pwr بواسطة x (كما في x(4.0, 2))، فإن this في العبارة السابقة كانت ستشير إلى x. كتابة العبارة دون استخدام this هي في الواقع مجرد اختصار.

فيما يلي المنشئ ( )pwr بأكمله مكتوبًا باستخدام المؤشر this :

```
int exp) { ،قاعدة مزدوجة)rwp::rwp

هذا->ب =الأساس؛ هذا->ه ؛-هذا= exp ؛إذا (exp == 0) val = 1؛ العودة؛ ل
(exp--؛ exp>0؛ هذا->val = -هذا->val * -هذا->val؛ >b؛-هذا->val

}
```

في الواقع، لن يكتب أي مبرمج C++ الدالة ()pwr كما هو موضح، لأنه لا يتم اكتساب أي شيء، والصيغة القياسية أسهل. مع ذلك، فإن مؤشر this مهم جدًا.

Machine Translated by Google

عندما يتم تحميل المشغلات بشكل زائد وعندما يتعين على وظيفة عضو استخدام مؤشر إلى الكائن الذي استدعاها.

تذكر أن هذا المؤشر يتم تمريره تلقائيًا إلى جميع الوظائف الأعضاء.
لذلك، يمكن أيضًا إعادة كتابة ( )get_pwrكما هو موضح هنا:

```
double get_pwr() { return this->val; }
```

في هذه الحالة، إذا تم استدعاء ( )get_pwrعلى النحو التالي:

```
y.get_pwr();
```

ثم سيشير هذا إلى الكائن .y
نقطتان أخيرتان حول هذا الموضوع. أولاً، الدوال الصديقة ليست أعضاءً في فئة، وبالتالي لا يُمرَّر إليها مؤشر . thisثانياً، الدوال الأعضاء الثابتة لا تحتوي على مؤشر . this

مؤشرات إلى الأنواع المشتقة

بشكل عام، لا يمكن لمؤشر من نوع واحد أن يشير إلى كائن من نوع مختلف. ومع ذلك، هناك استثناء مهم لهذه القاعدة يتعلق فقط بالفئات المشتقة.
للبدء، لنفترض وجود فئتين هما Bو .Dولنفترض أيضًا أن Dمشتقة من الفئة الأساسية .Bفي هذه الحالة، قد يشير مؤشر من نوع * Bأيضًا إلى كائن من نوع .D

بشكل عام، يمكن أيضًا استخدام مؤشر الفئة الأساسية كمؤشر إلى كائن من أي فئة مشتقة من تلك الفئة الأساسية.

مع أنه يُمكن استخدام مؤشر فئة أساسية للإشارة إلى كائن مُشتق، إلا أن العكس ليس صحيحًا. لا يُمكن لمؤشر من النوع * Dأن يُشير إلى كائن
من النوع .B علاوةً على ذلك، حتى لو أشرت إلى كائن مشتق باستخدام مؤشر من النوع الأساسي، فلن تتمكن من الوصول إلى أي من أعضاء الفئة المشتقة
(مع بعض الاستثناءات التي سنناقشها لاحقًا في هذا الفصل). ومع ذلك، يمكنك الوصول إلى أعضاء الفئة الأساسية. لتجاوز هذا القيد، يجب عليك تحويل مؤشر النوع الأساسي إلى مؤشر من النوع المشتق
مُشتق والحصول على وصول كامل إلى الفئة المُشتقة بأكملها).

فيما يلي برنامج قصير يوضح استخدام مؤشر أساسي للوصول إلى الكائنات المشتقة.

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

الفئة الأساسية { int i؛ عامة: void set_i(int num) { i=num؛ } int get_i() { return i؛ } }
```

Machine Translated by Google


```
                                { الفئة المشتقة: القاعدة العامة };

    void set_j(int num) { j=num; } int get_j() { return j; } :عام int j;



};

        int main() {

    القاعدة *bp; المشتقة
                d;

                                                    // bp = &d; يشير المؤشر الأساسي إلى الكائن المشتق

    bp->set_i(10); cout << bp->get_i() << // الوصول إلى الكائن المشتق باستخدام المؤشر الأساسي

                                                    " ";

                                                    */لن يعمل ما يلي. لا يمكنك الوصول إلى عناصر
                                                    فئة مشتقة باستخدام مؤشر فئة أساسية.

            bp->set_j(88); // خطأ cout << bp->get_j(); // خطأ

                                                    */العودة 0;

}
```

كما ترى، يتم استخدام مؤشر أساسي للوصول إلى كائن من فئة مشتقة.
مع ضرورة توخي الحذر، يُمكن تحويل مؤشر أساسي إلى مؤشر من النوع المُشتق للوصول إلى عنصر من الفئة المُشتقة عبر المؤشر الأساسي. على سبيل المثال، هذا كود C++ صالح:

```
//الآن أصبح الوصول مسموحًا به بسبب التحويل ((derived *)bp)->set_j(88); cout << ((derived *)bp)->get_j();
```

من المهم تذكر أن حساب المؤشر نسبي بالنسبة لنوعه الأساسي. لذلك، عندما يشير مؤشر أساسي إلى كائن مشتق، فإن زيادة قيمة المؤشر لا تؤدي إلى إشارته إلى الكائن التالي من النوع المشتق، بل سيشير إلى ما يعتقد أنه الكائن التالي من النوع الأساسي. هذا،

Machine Translated by Google

بالطبع، عادةً ما يُسبب هذا مشكلة. على سبيل المثال، هذا البرنامج، مع أنه صحيح لغويًا، إلا أنه يحتوي على هذا الخطأ.

```
//يحتوي هذا البرنامج على خطأ.
#تضمين <iostream>
استخدام مساحة الاسم ؛std

الفئة الأساسية ؛int i{ }عامة: { return i؛ } int get_i() { i=num؛ } (int
num) void set_i

};

الفئة المشتقة: القاعدة العامة {
void set_j(int num) {j=num;} int get_j() {return j;} عام: int j؛

};

int main() {

القاعدة ؛bp* المشتقة ؛[2]d

ب ب =د؛

د[0].set_i(1); د[1].set_i(2);

// ++bp->get_i() << cout بالنسبة إلى الأساس، وليس المشتق // ;()bp->get_i() << cout يتم عرض
قيمة غير مرغوب فيها

العودة ؛0
}
```

يعد استخدام المؤشرات الأساسية للأنواع المشتقة مفيدًا للغاية عند إنشاء وقت التشغيل
تعدد الأشكال من خلال آلية الوظائف الافتراضية (انظر الفصل .(17

Machine Translated by Google

مؤشرات أعضاء الفئة: تتيح لك لغة C++إنشاء نوع خاص من المؤشرات يُشير بشكل عام إلى عضو في فئة، وليس إلى مثيل محدد له في كائن. يُسمى هذا النوع من المؤشرات مؤشرًا إلى عضو في فئة أو مؤشر إلى عضو، اختصارًا. يختلف مؤشر العضو عن مؤشر C++العادي. بدلاً من ذلك، يُوفر مؤشر العضو إزاحة فقط إلى كائن من فئة العضو حيث يمكن العثور على هذا العضو. بما أن مؤشرات الأعضاء ليست مؤشرات حقيقية، فلا يمكن تطبيق . و <-عليها. للوصول إلى عضو في فئة مع وجود مؤشر إليه، يجب استخدام عاملي المؤشر إلى العضو الخاصين .* و .*<-وظيفتهما هي السماح لك بالوصول إلى عضو في فئة مع وجود مؤشر إلى هذا العضو.

وهنا مثال:

```
#تضمين <iostream>
استخدام مساحة الاسم :std

الصف cl{ عام: int val; int double_val() { val=i; } cl(int i) { إرجاع val+val; }

};

int main() {

إنشاء cl ob1(1)، ob2(2)؛ // مؤشر عضو البيانات int cl::*data; // مؤشر عضو الدالة int (cl::*func)(); //
كائنات

data = &cl::val; // الحصول على إزاحة val func = &cl::double_val; // الحصول على إزاحة double_val()

cout << "فيما يلي القيم: ";
cout << ob1.*data << " " << ob2.*data << "\n";

cout << "هنا يتم مضاعفتهم: " << (ob1.*func)() << " " << (ob2.*func)() << "\n";

العودة 0؛
}
```

Machine Translated by Google

في main( )، يُنشئ هذا البرنامج مؤشرين للأعضاء: data و .func لاحظ بعناية بنية كل إعلان. عند إعلان مؤشرات للأعضاء، يجب عليك تحديد الفئة واستخدام عامل حل النطاق. يُنشئ البرنامج أيضًا كائنات cl تُسمى ob1 و .ob2 وكما يوضح البرنامج، قد تُشير مؤشرات الأعضاء إلى دوال أو بيانات. بعد ذلك، يحصل البرنامج على عناوين val و .( )double_val وكما ذكرنا سابقًا، فإن هذه "العناوين" هي في الواقع مجرد إزاحات في كائن من نوع cl، وعند هذه النقطة سيتم العثور على val و . ( )double_val بعد ذلك، لعرض قيم val لكل كائن، يتم الوصول إلى كل منها من خلال البيانات. وأخيرًا، يستخدم البرنامج func لاستدعاء دالة double_val( ) الأقواس الإضافية ضرورية لربط عامل .* بشكل صحيح .

عند الوصول إلى عنصر من كائن باستخدام كائن أو مرجع (سنناقشه لاحقًا في هذا الفصل)، يجب استخدام عامل التشغيل .* . أما إذا كنت تستخدم مؤشرًا إلى الكائن، فيجب استخدام عامل التشغيل ، *<- كما هو موضح في هذه النسخة من البرنامج السابق:

```
#تضمين <iostream>
استخدام مساحة الاسم std؛

الصف cl {عام: int double_val() { val=i; } int val; }cl(int i) { إرجاع val+val; }

};

int main() {

إنشاء cl ob1(1)، ob2(2)؛ // مؤشر عضو الدالة int (cl::*func)(); // مؤشر عضو البيانات int cl::*data; //
كائنات cl *p1، *p2؛

// p1 = &ob1; الوصول إلى الكائنات من خلال المؤشر p2 = &ob2;

// data = &cl::val; الحصول على إزاحة val func = &cl::double_val; // الحصول على إزاحة double_val()

cout << "هذه هي القيم: " << p1->*data << " " << p2->*data << "\n";

cout << "هنا يتم مضاعفتهم: ";
```

Machine Translated by Google


```
<< (p1->*func)() << cout << (p2->*func)() << "\n";
                                        cout

    العودة 0؛
}
```

في هذا الإصدار، p1 وp2 هما مؤشران لكائنات من نوع cl. لذلك، يُستخدم عامل ->* للوصول إلى val و double_val().

تذكر أن مؤشرات العناصر تختلف عن مؤشرات العناصر المحددة في الكائن. انظر إلى هذا الجزء (افترض أن cl مُعلن كما هو موضح في البرامج السابقة):

```
int cl::*d؛ int *p؛ cl o؛

// p = &o.val هذا عنوان قيمة محددة

// d = &cl::val هذا هو إزاحة القيمة العامة
```

هنا، p هو مؤشر لعدد صحيح داخل كائن محدد . أما d فهو ببساطة إزاحة تشير إلى مكان وجود val في أي كائن من نوع cl.

بشكل عام، تُستخدم معاملات المؤشر إلى العضو في حالات خاصة. ولا تُستخدم عادةً في البرمجة اليومية.

تحتوي لغة C++ على ميزة مرتبطة بالمؤشر تُسمى "المرجع". والمرجع هو في الأساس مؤشر ضمني. هناك ثلاث طرق لاستخدام المرجع: كمعامل دالة، أو قيمة إرجاع دالة، أو كمرجع مستقل. سيتم شرح كل منها هنا.

معاملات المرجع: لعلّ أهم استخدام للمرجع هو تمكينك من إنشاء دوال تستخدم تمرير معاملات الاستدعاء بالمرجع تلقائيًا. كما هو موضح في الفصل السادس، يمكن تمرير الوسائط إلى الدوال بإحدى طريقتين: الاستدعاء بالقيمة أو الاستدعاء بالمرجع. عند استخدام الاستدعاء بالقيمة، تُمرّر نسخة من الوسيطة إلى الدالة. يُمرّر الاستدعاء بالمرجع عنوان الوسيطة إلى الدالة. افتراضيًا، تستخدم لغة C++ الاستدعاء بالقيمة، ولكنها توفر طريقتين لتحقيق تمرير معاملات الاستدعاء بالمرجع. أولًا، يمكنك تمرير مؤشر صراحةً إلى الوسيطة. ثانيًا،

Machine Translated by Google

يمكنك استخدام مُعامل مرجعي. في معظم الحالات، أفضل طريقة هي استخدام مُعامل مرجعي.

لفهم ما هي معلمة المرجع بشكل كامل ولماذا هي ذات قيمة، سنبدأ بمراجعة كيفية إنشاء استدعاء بالمرجع باستخدام معلمة مؤشر.
يقوم البرنامج التالي بإنشاء معلمة استدعاء بالمرجع يدويًا باستخدام مؤشر في الدالة المسماة neg( )، والذي يعكس إشارة المتغير الصحيح الذي يشير إليه وسيطته.

```
//قم بإنشاء استدعاء بالمرجع يدويًا باستخدام مؤشر.
#تضمين <iostream>
استخدام مساحة الاسم std؛

باطل neg(int *i);

int main() {

int x؛

س ؛10 =
cout<<"هَكَيْ هَوَ"<<س<<"\n";

neg(&x); cout << x << "\n";

العودة ؛0
}

باطل neg(int *i) {

*أنا =-*أنا؛
}
```

في هذا البرنامج، تأخذ الدالة neg( ) كمعامل مؤشرًا إلى العدد الصحيح الذي ستعكس إشارته. لذلك، يجب استدعاء الدالة neg( ) صراحةً بعنوان x. علاوةً على ذلك، يجب استخدام عامل * داخل الدالة neg( ) للوصول إلى المتغير الذي يشير إليه . أهذه الطريقة تعمل على استدعاء مرجعي باستخدام مجموعة C الفرعية؟ نعم، ولكن هناك طريقة أفضل بكثير في C++ وهي الطريقة التي يستخدمها الغالبية العظمى من مبرمجي C++ عند الحاجة إلى استدعاء مرجعي.

لإنشاء معلمة مرجعية، ضع علامة & قبل اسم المعلمة . بالنسبة لـ
على سبيل المثال، إليك كيفية إعلان neg( ) مع إعلان i كمعلمة مرجعية:

```
void neg(int &i);
```

Machine Translated by Google

عمليًا، يُؤدي هذا إلى أن يصبح iاسمًا آخر لأي وسيطة تُستدعى بها الدالة . ( )negأي عمليات تُطبق على iتؤثر فعليًا على الوسيطة المُستدعاة. تقنيًا، iهو مؤشر ضمني يُشير تلقائيًا إلى الوسيطة المُستخدمة في استدعاء .( )negبمجرد تحويل iإلى مرجع، لم يعد من الضروري (أو حتى القانوني) تطبيق المُعامل . *بدلًا من ذلك، في كل مرة يُستخدم فيها i ، iيكون مرجعًا ضمنيًا للوسيطة، وأي تغييرات تُجرى عليه تؤثر على الوسيطة .

علاوةً على ذلك، عند استدعاء ،()negلم يعد من الضروري (أو القانوني) إضافة عامل &قبل اسم الوسيطة . بدلاً من ذلك، يقوم المُجمِّع بذلك تلقائيًا. إليك النسخة المرجعية للبرنامج السابق:

```
//استخدم معلمة مرجعية.
#تضمين <iostream>
استخدام مساحة الاسم ؛std

void neg(int &i); // iأنا الآن مرجع

int main() {
  int x؛
  س ؛10 =
  cout؛"تَفَيِّ هَوَّ"
  neg(x); // لم تعد هناك حاجة إلى عامل "\n" << x << & cout
  العودة ؛0
}

void neg(int &i) {
  i // ؛i- = iهو الآن مرجع، لا حاجة إلى *
}
```

للمراجعة: عند إنشاء مُعامل مرجعي، فإنه يُشير تلقائيًا (ضمنيًا) إلى الوسيطة المُستخدمة لاستدعاء الدالة. لذلك، في البرنامج السابق، العبارة

```
أنا =-أنا؛
```

يعمل فعليًا على ،xوليس على نسخة منه . لا حاجة لتطبيق عامل &على أي وسيطة. أيضًا، داخل الدالة، يُستخدم مُعامل المرجع مباشرةً دون الحاجة إلى

Machine Translated by Google

الحاجة إلى تطبيق عامل . *بشكل عام، عند تعيين قيمة لمرجع،
في الواقع، أنت تقوم بتعيين تلك القيمة للمتغير الذي يشير إليه المرجع.
داخل الدالة، ليس من الممكن تغيير ما هو معامل المرجع
يشير إلى. أي عبارة مثل

```
اي+:+
```

داخل ()negيزيد قيمة المتغير المستخدم في الاستدعاء. لا يسبب
للإشارة إلى موقع جديد.
هذا مثال آخر. يستخدم هذا البرنامج معلمات مرجعية لتبادل
قيم المتغيرات التي يتم استدعاؤها بها. دالة ( )swapهي المثال الكلاسيكي
من تمرير معلمات الاستدعاء بالمرجع.

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

void swap(int &i, int &j);

int main()
{
int a, b, c, d;

أ ؛1 =
ب ؛2 =
ج ؛3 =
د ؛4 =

cout << "a and b: " << أ؛ "" << "\n"; >>ب

// swap(a, b); لا حاجة لعامل &
cout << "a and b: " << أ؛ "" << "\n"; >>ب

d: " وcout ؛ج؛ "" << "\n"; >>د
مبادلة(ج، د)؛
d: " وcout ؛ج؛ "" << "\n"; >>د

العودة ؛0
}

void swap(int &i, int &j)
{
int t؛

t = i; // لا * مطلوب مشغل
```

Machine Translated by Google


```
    i = j; j = t;
}
```

يعرض هذا البرنامج ما يلي:

```
أ و ب: 1 2
أ و ب: 2 1
ج و د: 3 4
ج و د: 4 3
```


تمرير المراجع إلى الكائنات

في الفصل الثاني عشر، شرحنا أنه عند تمرير كائن كمعامل إلى دالة، تُنشأ نسخة منه. عند انتهاء الدالة، يُستدعى مُدمر النسخة. أما عند التمرير بالمرجع، فلا تُنشأ أي نسخة من الكائن. هذا يعني أنه لا يُدمر أي كائن يُستخدم كمعامل عند انتهاء الدالة، ولا يُستدعى مُدمر المعامل. على سبيل المثال، جرّب هذا البرنامج:

```
#تضمين <iostream>
استخدام مساحة الاسم std;

cl { int id; public: int i; cl(int i); ~cl(); void neg(cl &o) { oi = -oi; } // الفئة لم يتم إنشاء مؤقت };

cl::cl(int num)
{
id = num; "إنشاء"cout << << "\n"; >>رقم

}

cl::~cl()
{
<< معرف"تدمير" <<
}
```

Machine Translated by Google


```
int main() {

    cl o(1)؛

10؛ o.neg(o)؛
    oi =

        cout << oi << "\n";

العودة ؛0
    }
```

وهنا هو الناتج من هذا البرنامج:

```
بناء 1
-10
تدمير 1
```

كما ترى، يتم استدعاء مُدمر الدالة clمرة واحدة فقط. لو تم تمرير الدالة oبالقيمة، لكان قد تم إنشاء كائن ثانٍ داخل الدالة ،( )negوكان سيتم استدعاء المُدمر مرة أخرى عند تدمير ذلك الكائن عند انتهاء الدالة . ( )neg

كما يوضح الكود الموجود داخل ، ( )negعندما تقوم بالوصول إلى أحد أعضاء الفصل من خلال كمرجع، استخدم عامل النقطة. عامل السهم مخصص للاستخدام مع المؤشرات فقط.

عند تمرير المعلمات عن طريق المرجع، تذكر أن التغييرات التي تطرأ على الكائن داخل
الوظيفة تؤثر على الكائن المستدعي.
نقطة أخرى: تمرير جميع الكائنات، باستثناء أصغرها، بالرجوع إليها أسرع من تمريرها بالقيمة. عادةً ما تُمرَّر الوسائط على المكدس. وبالتالي، تستغرق الكائنات الكبيرة عددًا كبيرًا من دورات وحدة المعالجة المركزية لتُدفع إلى المكدس وتُخرج منه.

إرجاع المراجع: قد تُرجع الدالة مرجعًا. وهذا يُتيح استخدام الدالة على يسار جملة الإسناد! على سبيل المثال، لننظر إلى هذا البرنامج البسيط:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

char &replace(int i); // إرجاع مرجع
```

Machine Translated by Google


```
                                ;"مرحبا"char s[80] =

int main() {


                                                    Hello بعد Xللمسافة تعيين replace(5) = 'X'; //

    cout << s;

    0؛ العودة
}

char &replace(int i) {

    s[i]؛ العودة
}
```


يستبدل هذا البرنامج المسافة بين Helloو There بعلامة .Xأي أن البرنامج يعرض .HelloXthereانظر كيف يتم ذلك. أولًا، تُعلن دالة ( )replaceعلى أنها تُرجع مرجعًا إلى حرف. عند كتابة ، ( )replaceتُرجع مرجعًا إلى عنصر sالمُحدد بمُعامله .ثم يُستخدم المرجع المُعاد بواسطة ( )replaceفي ( )mainلتعيين الحرف Xلهذا العنصر.

أحد الأشياء التي يجب أن تكون حذرًا بشأنها عند إرجاع المراجع هو أن الكائن
إن الإشارة إلى ما لا يخرج عن النطاق بعد انتهاء الوظيفة.

المراجع المستقلة

من أكثر استخدامات المراجع شيوعًا تمرير وسيطة باستخدام استدعاء مرجعي، والعمل كقيمة إرجاع من دالة. مع ذلك، يمكنك إعلان مرجع يكون ببساطة متغيرًا. يُسمى هذا النوع من المراجع مرجعًا مستقلًا.

عند إنشاء مرجع مستقل، كل ما تُنشئه هو اسم آخر لكائن. يجب تهيئة جميع المراجع المستقلة عند إنشائها. والسبب واضح. فباستثناء التهيئة، لا يمكنك تغيير الكائن الذي يشير إليه متغير مرجعي. لذلك، يجب تهيئة الكائن عند إعلانه. (في لغة ،C++التهيئة عملية منفصلة تمامًا عن التعيين).

يوضح البرنامج التالي مرجعًا مستقلاً:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std
```

Machine Translated by Google


```
int main()
{
int a;
int &ref = a; // مرجع مستقل

cout << "a = 10: " << a << "\n";
cout << "ref = 100: " << ref << "\n";

int b = 19;
ref = b; // هذا يضع قيمة b في a
cout << "ref = " << ref << "\n";

// ref--; هذا يقلل من
//لا يؤثر على ما يشير إليه المرجع

cout << "ref = " << ref << "\n";

return 0: العودة
}
```

يعرض البرنامج هذا الإخراج:

```
10 10
100 100
19 19
18 18
```

في الواقع، المراجع المستقلة لها قيمة حقيقية قليلة لأن كل واحدة منها، حرفيًا،
مجرد اسم آخر لمتغير آخر. وجود اسمين لوصف نفس الكائن
من المرجح أن يؤدي ذلك إلى إرباك برنامجك وليس تنظيمه.

المراجع إلى الأنواع المشتقة

على غرار الموقف كما هو موضح للمؤشرات سابقًا، يمكن أن يكون مرجع الفئة الأساسية
يُستخدم للإشارة إلى كائن من فئة مشتقة. التطبيق الأكثر شيوعًا لهذا هو
موجودة في معلمات الدالة. يمكن لمعلمة مرجعية للفئة الأساسية استقبال كائنات من
الفئة الأساسية وكذلك أي نوع آخر مشتق من تلك القاعدة.

Machine Translated by Google

القيود المفروضة على المراجع

هناك عدد من القيود التي تنطبق على المراجع. لا يمكنك الرجوع إلى مرجع آخر. بمعنى آخر، لا يمكنك الحصول على عنوان مرجع. لا يمكنك إنشاء مصفوفات من المراجع. لا يمكنك إنشاء مؤشر إلى مرجع. لا يمكنك الرجوع إلى حقل بت.

يجب تهيئة متغير مرجعي عند إعلانه ما لم يكن عضوًا
لفئة، أو معلمة دالة، أو قيمة إرجاع. يُحظر استخدام مراجع فارغة.

مسألة أسلوب

عند إعلان متغيرات المؤشر والمرجع، يستخدم بعض مبرمجي C++أسلوب ترميز فريدًا يربط الرمز *أو &باسم النوع وليس المتغير. على سبيل المثال، إليك إعلانان متكافئان وظيفيًا:

```
int& p; // & مرتبط بنوع int& p; // & مرتبط بالمتغير
```

```
int *أ، ب؛
```

هنا، يُعلن عن bكعدد صحيح (وليس مؤشرًا صحيحًا) لأنه، كما هو محدد في بناء جملة C++، عند استخدامه في إعلان، يرتبط *(أو &بالمتغير الذي يسبقه، وليس بالنوع الذي يليه. تكمن مشكلة هذا الإعلان في أن الرسالة البصرية توحي بأن كلاً من a وbنوعان من المؤشرات، مع أن aفقط هو مؤشر في الواقع. هذا الالتباس البصري لا يُضلل مبرمجي C++المبتدئين فحسب، بل يُضلل أيضًا المحترفين القدامى أحيانًا.

من المهم أن نفهم أنه فيما يتعلق بمترجم C++، فهو لا

مشغلات التخصيص الديناميكي في C++

توفر لغة C++عاملي تخصيص ديناميكيين: جديد وحذف . يُستخدم هذان العاملان لتخصيص الذاكرة وتحريرها أثناء التشغيل. يُعد التخصيص الديناميكي جزءًا مهمًا.

Machine Translated by Google

في معظم برامج العالم الحقيقي. كما هو موضح في الجزء الأول، تدعم لغة C++أيضًا دوال تخصيص الذاكرة الديناميكية، ( )mallocو ( )freeتم تضمين هذه الدوال لضمان التوافق مع لغة .Cمع ذلك، بالنسبة لأكواد ،C++يُنصح باستخدام عاملي newو deleteلما لهما من مزايا عديدة.

يُخصِّص مُشغِّل "جديد" الذاكرة ويُعيد مؤشرًا إلى بدايتها. يُحرِّر مُشغِّل "حذف" الذاكرة المُخصَّصة سابقًا باستخدام "جديد". الشكلان العامان لـ "جديد" و "حذف" موضحان هنا:

p_var = نوع جديد؛ حذف p_var؛

هنا، p_varهو متغير مؤشر يستقبل مؤشرًا إلى الذاكرة كبير بما يكفي لحمل عنصر من نوع النوع .

بما أن الكومة محدودة، فقد تُستنفد. إذا لم تكن هناك ذاكرة كافية لتلبية طلب التخصيص، فسيفشل أمر newوسينشأ استثناء bad_alloc . يُعرّف هذا الاستثناء في العنوان <new>. يجب على برنامجك معالجة هذا الاستثناء واتخاذ الإجراء المناسب في حال حدوث أي عطل. (يُشرح التعامل مع الاستثناءات في الفصل 19.) إذا لم يُعالج برنامجك هذا الاستثناء، فسيتم إنهاء البرنامج.

إجراءات الأمر "جديد" عند الفشل، كما وُصف سابقًا، مُحددة في معيار C++. تكمن المشكلة في أن جميع المُجمّعات، وخاصةً القديمة منها، لا تُطبّق الأمر "جديد" وفقًا لمعيار C++. عند اختراع C++ لأول مرة، كانت تُعيد الأمر "جديد" "قيمة "لا شيء" عند الفشل. لاحقًا، تغيّر هذا الوضع بحيث تُسبّب "جديد" استثناءً عند الفشل. أخيرًا، اتُخذ قرار بأن يُولّد "جديد" استثناءً افتراضيًا، ولكن يُمكن إرجاع مؤشر "لا شيء" كخيار. وبالتالي، طُبّقت "جديد" بشكل مختلف، وفي أوقات مختلفة، من قِبل مُصنّعي المُجمّعات. على الرغم من أن جميع المُجمّعات ستُطبّق الأمر "جديد" في النهاية وفقًا لمعيار C++، إلا أن الطريقة الوحيدة حاليًا لمعرفة الإجراء الدقيق للأمر "جديد" عند الفشل هي مراجعة وثائق المُجمّع.

بما أن لغة C++القياسية تنص على أن الأمر newيُنشئ استثناءً عند الفشل، فهذه هي طريقة كتابة الكود في هذا الكتاب. إذا كان مُجمِّعك يُعالج فشل التخصيص بشكل مختلف، فستحتاج إلى إجراء التغييرات المناسبة.

فيما يلي برنامج يخصص الذاكرة لحمل عدد صحيح:

```
#تضمين <iostream>
#تضمين <جديد>
استخدام مساحة الاسم std؛

int main() {

  int *p؛

يحاول {
```

Machine Translated by Google


```
int } catch (bad_alloc xa) { لـ تخصيص مساحة p = new int; //

return 1; ;"n\فشل التخصيص"cout <<

}

*ص ؛100 =

cout<<"ص<<"Ah";
>>ص"هي"Ih"يمة"<<cout

حذف ص؛

العودة ؛0
}
```

سي++

يقوم هذا البرنامج بتعيين عنوان في الكومة إلى pكبير بما يكفي لاحتواء
عدد صحيح. ثم يُعيّن القيمة ..التلك الذاكرة ويعرض محتوياتها على الشاشة. وأخيرًا، يُحرّر الذاكرة المُخصّصة ديناميكيًا. تذكّر، إذا طبّق مُجمّعك الأمر newبحيث يُعيد القيمة nullعند الفشل، فيجب عليك تعديل البرنامج السابق بشكل مناسب.

يجب استخدام مُعامل الحذف فقط مع مؤشر صالح مُخصص مسبقًا باستخدام أمر "جديد ."استخدام أي نوع آخر من المؤشرات مع الحذف غير مُعرّف، ومن المُرجح أن يُسبب مشاكل خطيرة، مثل تعطل النظام.

على الرغم من أن دالة newو deleteتؤديان وظائف مشابهة لدالتي ()mallocو ،()freeإلا أنهما تتمتعان بمزايا عديدة. أولًا، تُخصص دالة newتلقائيًا ذاكرة كافية لاستيعاب كائن من النوع المحدد. لا حاجة لاستخدام مُعامل . sizeofولأن الحجم يُحسب تلقائيًا، فإنه يُلغي أي احتمال للخطأ في هذا الصدد. ثانيًا، تُرجع دالة newتلقائيًا مؤشرًا من النوع المحدد. لا حاجة لاستخدام تحويل نوع صريح كما هو الحال عند تخصيص الذاكرة باستخدام .()mallocأخيرًا، يُمكن تحميل دالة newو deleteبشكل زائد، مما يسمح لك بإنشاء أنظمة تخصيص مُخصصة.

على الرغم من عدم وجود قاعدة رسمية تنص على ذلك، فمن الأفضل عدم خلط الجديد مع الحذف مع ()mallocو ()freeفي نفس البرنامج. لا يوجد ضمان للتوافق بينهما.

تهيئة الذاكرة المخصصة: يمكنك تهيئة الذاكرة المخصصة إلى قيمة معروفة بإضافة مُهيئ بعد اسم النوع في جملة "جديد" . هذا هو الشكل العام لـ "جديد" عند تضمين مُهيئ:

```
p_var = new var_type (initializer);
```

Machine Translated by Google

بالطبع، يجب أن يكون نوع المُبدئ متوافقًا مع نوع البيانات التي يتم تخصيص الذاكرة لها.

يعطي هذا البرنامج العدد الصحيح المخصص قيمة أولية قدرها 87:

```
#تضمين <iostream>
#تضمين <جديد>
استخدام مساحة الاسم ؛std

int main() {

  int *p؛

                    حاول // ;(p = new int (87 }قم بالتهيئة إلى 87

                } catch (bad_alloc xa) {
                  return 1; ;"n\فشل التخصيص"cout <<

}

                                                  cout<<ص<<"هذا";
                                                  >>"ما القيمة "<<*p;

  حذف ص؛

  العودة ؛0
  }
```

تخصيص المصفوفات

يمكنك تخصيص المصفوفات باستخدام newباستخدام هذا النموذج العام:

p_var = نوع المصفوفة الجديد [الحجم]؛

هنا، يحدد الحجم عدد العناصر الموجودة في المصفوفة.
لتحرير مجموعة، استخدم هذا النموذج من الحذف:

حذف ؛p_var [ ]

هنا، [ ]يخبرك بحذف مجموعة يتم تحريرها.
على سبيل المثال، يقوم البرنامج التالي بتخصيص مصفوفة عدد صحيح مكونة من 10عناصر.

Machine Translated by Google


```
#تضمين <iostream>
#تضمين <جديد>
استخدام مساحة الاسم ؛std

int main() {

    int *p، i؛

                    حاول // ؛[10] int new = p { تخصيص 10مصفوفة عددية صحيحة

                } catch (bad_alloc xa) {
                    return 1; ;"n\فشل التخصيص"cout <<

}

                i<10؛ i++ ) ؛0=i)ل
                    p[i] = i؛

    i++) cout << p[i] << بالنسبة إلى
                (i = 0؛ i < 10؛     " ";

                        حذف // ؛[] pتحرير المصفوفة

        0؛ العودة
            }
```

لاحظ عبارة الحذف . كما ذكرنا سابقًا، عند تحرير مصفوفة مُخصصة باستخدام new ، يجب إعلام الحذف بأنه يتم تحرير مصفوفة باستخدام .[ ]
(كما سترى في القسم التالي، يعد هذا مهمًا بشكل خاص عند تخصيص مجموعات من الكائنات.)

ينطبق قيد واحد على تخصيص المصفوفات: لا يجوز منحها قيمًا أولية.
وهذا يعني أنه لا يجوز لك تحديد مُبدئ عند تخصيص المصفوفات.

تخصيص الكائنات

يمكنك تخصيص الكائنات ديناميكيًا باستخدام الأمر "جديد". عند القيام بذلك، يُنشأ كائن ويُعاد إليه مؤشر. يعمل الكائن المُنشأ ديناميكيًا كأي كائن آخر. عند إنشائه، يُستدعى مُنشئه (إن وُجد). عند تحرير الكائن، يُنفَّذ مُدمره.

Machine Translated by Google

هذا برنامج قصير يُنشئ فئة تُسمى balance ، balanceتربط اسم الشخص برصيد حسابه. داخل main( )، يُنشأ كائن من نوع balanceديناميكيًا.

```
#تضمين <iostream>
#تضمين <جديد>
#include <cstring>باستخدام مساحة الاسم std؛

{ توازن الطبقة
char name[80]؛ public: void set(double n, char *s) {
double cur_bal؛

s)؛ ،الاسم)ypcrtscur_bal = n؛

}

void get_bal(double &n, char *s) {
؛(الاسم)؛ln = cur_bal؛ strcpy(s،

}
};

int main() {

؛مزدوج؛p*؛ char s[80]؛ n التوازن

catch (bad_alloc xa) { {؛رصيد جديد}p = حاول

return 1; ;"n\فشل التخصيص"cout <<

}

;("رالف ويلسون"p->set(12387.87,

p->get_bal(n, s);

" :هوcout << s << " ؛رصيد
```

Machine Translated by Google


```
        cout << "\n";

    حذف ص؛

    العودة 0؛
}
```

نظرًا لأن p يحتوي على مؤشر إلى كائن، يتم استخدام عامل السهم للوصول إلى أعضاء الكائن.

كما هو مذكور، قد تحتوي الكائنات المخصصة ديناميكيًا على منشئين ومدمرين.
يمكن أيضًا تحديد معلمات المُنشئين. راجع هذه النسخة من البرنامج السابق:


```
#تضمين <iostream>
#تضمين <جديد>
#include <cstring>باستخدام مساحة
std؛ الاسم

توازن الطبقة {
n، ضعف)ecnalab :عام[80]؛ اسم الحرف cur_bal؛ ضعف
s)؛ الاسم، (ypcrts*s) { cur_bal = n؛ حرف

~التوازن() {}

"; cout << name << "\n"; تدمير"cout <<

} void get_bal(double &n, char *s) {

strcpy(s، ln = cur_bal؛ الاسم)؛

}
};

int main() {

char s[80]؛ n التوازن
p؛ مزدوج*؛
```

Machine Translated by Google


```
{ p = new balance (12387.87, "Ralph Wilson"); } catch (bad_alloc xa) { //يستخدم هذا الإصدار مُبدئًا
try

return 1; ;"n\فشل التخصيص"cout <<

}

p->get_bal(n, s);

" :هوcout << s << " ؛رصيد
cout << "\n";

حذف ص؛

العودة ؛0
}
```

لاحظ أن معلمات منشئ الكائن يتم تحديدها بعد اسم النوع، تمامًا كما هو الحال في أنواع أخرى من عمليات التهيئة الأولية.

يمكنك تخصيص مصفوفات من الكائنات، ولكن هناك مشكلة واحدة. بما أنه لا يمكن لأي مصفوفة مخصصة بواسطة الأمر "جديد" أن تحتوي على مُهيئ، يجب عليك التأكد من أنه إذا كانت الفئة تحتوي على مُنشئات، فسيكون أحدها بدون معلمات. إذا لم تفعل ذلك، فلن يعثر مُترجم C++ على مُنشئ مُطابق عند محاولة تخصيص المصفوفة، ولن يُترجم برنامجك.

في هذا الإصدار من البرنامج السابق، يتم تخصيص مجموعة من كائنات التوازن ، ويتم استدعاء المنشئ الخالي من المعلمات.

```
#تضمين <iostream>
#تضمين <جديد>
#include <cstring>باستخدام مساحة الاسم
std؛

توازن الطبقة {
ضعف cur_bal؛ اسم الحرف ؛[80]عام: ecnalab(ضعف ،nحرف
s)؛ ،الاسم)ypcrts*s) { cur_bal = n؛

// balance() {} {منشئ بدون معلمات
```

Machine Translated by Google


```
                    { ()التوازن~
                                    "; cout << name << "\n"; cout <<"تدمير
                                                } void set(double n, char *s) {
                                                    s)؛ ،الاسم)ypcrtscur_bal = n؛
                                                } void get_bal(double &n, char *s) {
                                                    ؛)الاسمln = cur_bal؛ strcpy(s،
    }
};

    int main() {

  char s[80]؛ n التوازن
      int i؛ ؛p*مزدوج؛

                                                    حاول = p}رصيد جديد // ؛[3]تخصيص المصفوفة بأكملها

                                    } catch (bad_alloc xa) {
                                        return 1; ;"n\فشل التخصيص"cout <<

    }

          p[2].set(-11.23, "IM Overdrawn"); //لاحظ استخدام النقاط وليس مشغلي الأسهم
              p[0].set(12387.87, "Ralph Wilson"); p[1].set(144.00, "AC Conners");



                                    i<3؛ i++) { ؛0=i)لـ
                                    p[i].get_bal(n, s);

                                                        رصيد؛ " << s << "هو: "cout
                                                        cout << "\n";
                                                        }

    0؛ العودة ؛ص[] حذف

}
```

Machine Translated by Google

يظهر الناتج من هذا البرنامج هنا.

```
رصيد رالف ويلسون هو: 12387.9
رصيد AC Conners هو: 144
رصيد السحب على المكشوف في IM هو: -11.23
تدمير IM Overdraw تدمير AC Conners تدمير Ralph Wilson
```

أحد الأسباب التي تجعلك بحاجة إلى استخدام نموذج الحذف [ ] عند حذف مجموعة من الكائنات المخصصة ديناميكيًا هو أنه يمكن استدعاء المدمر لكل كائن في المجموعة.

البديل الشمالي

في لغة C++ القياسية، من الممكن استخدام new لإرجاع قيمة null بدلاً من طرح استثناء عند فشل التخصيص. يُعد هذا الشكل من new مفيدًا للغاية عند تجميع شيفرة قديمة باستخدام مُجمِّع C++ حديث. كما أنه مفيد عند استبدال استدعاءات malloc() بـ new. (هذا شائع عند تحديث شيفرة C إلى C++). يظهر هذا الشكل من new هنا:

```
p_var = نوع جديد(لا ترمي) ؛
```

هنا، p_var هو متغير مؤشر من نوع. يعمل خيار nothrow من new بنفس طريقة النسخة الأصلية من new من سنوات سابقة. ولأنه يُرجع قيمة null عند الفشل، يُمكن إضافته إلى الكود القديم دون الحاجة إلى معالجة الاستثناءات. مع ذلك، بالنسبة للكود الجديد، تُوفر الاستثناءات بديلاً أفضل. لاستخدام خيار nothrow ، يجب تضمين رأس <new>.

يوضح البرنامج التالي كيفية استخدام البديل الجديد . (nothrow)

```
//إظهار إصدار notrow من الجديد.
#تضمين <iostream>
#تضمين <جديد>
استخدام مساحة الاسم ؛std

int main() {

  int *p، i؛

// p = new(nothrow) int[32]; if(!p) { cout << "فشل التخصيص.";\n" }
```

Machine Translated by Google


```
    العودة 1؛
}

(i = 0؛ i < 32؛ i++) p[i] = i؛ بالنسبة إلى

(i = 0؛ i <32؛ i++) cout << p[i] << بالنسبة إلى

حذف // p؛ []تحرير الذاكرة

العودة 0؛
}
```


كما يوضح هذا البرنامج، عند استخدام نهج nothrow ، يجب عليك التحقق من المؤشر الذي تم إرجاعه بواسطة new بعد كل طلب تخصيص.

نموذج التوظيف الجديد

هناك شكل خاص من new، يُسمى شكل التعيين، يُستخدم لتحديد طريقة بديلة لتخصيص الذاكرة. وهو مفيد بشكل أساسي عند زيادة تحميل مُشغل new في ظروف خاصة. إليك شكله العام:

p_var = نوع جديد (قائمة وسيطة) ؛

هنا، arg-list عبارة عن قائمة منفصلة بفاصلة من القيم التي يتم تمريرها إلى نموذج محمل جديد.

Machine Translated by Google

تم ترك هذه الصفحة فارغة عمدًا

Machine Translated by Google

The Complete Reference



الفصل 14

التحميل الزائد للوظيفة، منشئو النسخ والحجج الافتراضية

جميع الحقوق محفوظة ٢٠٠٣ ©لشركة ماكجرو هيل. انقر هنا للاطلاع على شروط الاستخدام.

Machine Translated by Google

الوسائط. يُعدّ التحميل الزائد للدوال أحد الجوانب الأساسية في لغة برمجة C++. فهو لا يدعم تعدد أشكال وقت التجميع فحسب، بل يُضيف أيضًا مرونة وسهولة في الاستخدام. من خلال تحميل الدوال المتعددة بأنواع مختلفة من الوسائط، يُمكن للمبرمجين إنشاء دوال متسقة ولعلّ أهم أشكال المُنشئ المُحمّل بالتحميل الزائد هو مُنشئ النسخ. ترتبط الوسائط الافتراضية ارتباطًا وثيقًا بالتحميل الزائد للدوال، حيث يُمكن أن تُوفر أحيانًا بديلًا عن التحميل الزائد للدوال.

التحميل الزائد للوظائف

التحميل الزائد للوظيفة هو عملية استخدام نفس الاسم لوظيفتين أو أكثر.
يكمن سرّ التحميل الزائد في أن كل إعادة تعريف للدالة يجب أن تستخدم إما أنواعًا مختلفة من المعاملات أو عددًا مختلفًا منها. ومن خلال هذه الاختلافات فقط، يعرف المُجمّع أي دالة يجب استدعاؤها في أي حالة.

على سبيل المثال، يقوم هذا البرنامج بتحميل ( )myfuncبشكل زائد عن الحد باستخدام أنواع مختلفة من المعلمات.

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

// ; (int i) myfuncتختلف هذه في أنواع المعلمات ;(double i) myfunc

int main() {

// ; (10) myfuncيستدعي cout << myfunc(i) << "\n"; // ; (5.4) myfuncيستدعي cout << myfunc(i) << "\n";

العودة ؛0
}

وظيفة myfuncمزدوجة iمزدوجة) {

العودة أنا؛
}

int myfunc(int i) {

العودة أنا؛
}
```

Machine Translated by Google

يقوم البرنامج التالي بتحميل ( )myfuncباستخدام عدد مختلف من المعلمات:

```
#تضمين <iostream>
استخدام مساحة الاسم ؛std

int myfunc(int i); // int myfunc(int i, int j)تختلف هذه في عدد المعلمات ;

int main() {

cout << "myfunc(10) يستدعي" << myfunc(10) << '\n'; // myfunc(4, 5)يستدعي ؛

cout << "myfunc(4, 5) يستدعي" << myfunc(4, 5) << '\n'; // myfunc(int i, int j)يستدعي

العودة ؛0
}

int myfunc(int i) {

العودة أنا؛
}

int myfunc(int i, int j) {

العودة ؛j*i
}
```

كما ذكرنا، فإن النقطة الأساسية حول التحميل الزائد للوظائف هي أن الوظائف يجب أن تكون
تختلف في أنواع و/أو عدد المعاملات. لا يمكن تحميل دالتين تختلفان فقط في أنواع الإرجاع. على سبيل المثال، هذه محاولة غير
صالحة لتحميل دالة :( )myfunc

```
int myfunc(int i); // float myfunc(int i)خطأ، لا يمكن التحميل الزائد فقط على نوع الإرجاع
```

في بعض الأحيان، قد يبدو أن إعلانين لوظيفتين مختلفين، بينما في الواقع هما ليسا مختلفين.
على سبيل المثال، ضع في اعتبارك الإعلانات التالية.

```
void f(int *p); void f(int p[]); // خطأ، p* هو نفسه p[]
```

تذكر، بالنسبة للمترجم، p* هو نفسه .[ ]pلذلك، على الرغم من أن النموذجين الأوليين يبدو أنهما يختلفان في أنواع معلماتهما، إلا
أنهما في الواقع ليسا كذلك.